

Bearing Early Fault Warning

Unsupervised anomaly detection and early-warning on the NASA/IMS run-to-failure bearing dataset

Technical Report

Ecem Aydemir

October 2026

Abstract

Rolling-element bearings are among the most common causes of failure in rotating machinery, and failure labels are almost never available in practice. This report describes a complete, reproducible early-warning system that learns what *healthy* vibration looks like from the first part of a machine's life and raises a debounced alarm when a bearing drifts away from it. The system reduces each one-second, 20 kHz vibration snapshot to 18 physically meaningful condition indicators (time-domain statistics, band energies and envelope-spectrum energy at the bearing defect frequencies), compares four detectors (an RMS threshold baseline, Mahalanobis distance, Isolation Forest and an LSTM autoencoder) and adds an envelope-spectrum diagnosis of the likely defect. On the development test (IMS test 2) every detector raises a persistent alarm on the failing bearing about **74 hours (45 % of the run) before the end of the test** and correctly identifies an **outer-race** defect. On the two held-out tests the results expose a clear trade-off between lead time and false alarms, and an honest limitation: operating-condition changes after machine stops look like anomalies. The work is delivered as an open-source Python package with a one-command pipeline, unit tests, a walkthrough notebook and an interactive Streamlit dashboard.

Contents

How to Read This Report	3
1 Introduction	4
1.1 Problem	4
1.2 Approach in one paragraph	4
1.3 Contributions	4
1.4 Relation to an earlier attempt	5
2 Background	5
2.1 Rolling-element bearings and their defects	5
2.2 Vibration condition indicators	7
2.3 Unsupervised anomaly detection	9
3 Dataset	10
4 Tools and environment	11

5	Methodology	12
5.1	Feature extraction	12
5.2	Signal-level illustration	14
5.3	Preprocessing and data splits	17
5.4	Detectors	18
5.5	From scores to alarms	20
5.6	Diagnosis	21
5.7	Evaluation protocol	22
5.8	Design decisions	22
6	Results	23
6.1	Headline results	23
6.2	Test 2 (development set)	25
6.3	Test 1 (held out)	29
6.4	Test 3 (held out)	31
6.5	Summary of findings	33
7	Interactive dashboard	33
8	Software engineering and reproducibility	35
8.1	Repository layout	35
8.2	Configuration	36
8.3	Running	36
8.4	Testing	37
8.5	Version control	38
9	Discussion and limitations	38
10	Conclusion and future work	39
A	Glossary of Key Terms	39

How to Read This Report

The story in a few sentences. Think of a family doctor who listens to your heartbeat at every visit. After a few visits the doctor knows what *your* normal heartbeat sounds like. One day it sounds slightly different, and the doctor sends you for a check-up long before anything serious happens. This project builds such a “doctor” for machines.

Almost every machine with a turning shaft (a motor, a pump, a fan, a car wheel) has *bearings*: small rings with balls or rollers inside that let the shaft turn smoothly. Bearings wear out, and a worn bearing usually announces itself before it breaks, just as a car wheel bearing starts to hum for weeks before it fails. The problem is that the change is tiny at first and easy to miss.

Our system listens to the vibration (the shaking) of four bearings on a test machine. During the first quarter of the machine’s life, when everything is assumed to be fine, it learns what “normal” vibration looks like. After that it keeps listening, and raises an alarm when the vibration of one bearing moves away from its normal pattern and *stays* away. Finally it tries to say which part of the bearing is damaged, by listening for the rhythm of the small knocks that a damaged spot produces.

Why it matters. An unexpected breakdown stops production and can damage other parts of the machine. Replacing bearings “just in case” wastes money, because many of them still had a long life ahead. An early warning lets a maintenance team plan the repair for a convenient moment. The difficulty is that real factories almost never have recorded examples of how *their* machine sounds when it is failing. So the system must learn from healthy data only, and recognise anything that is “not normal”.

The main result in one sentence. On the cleanest of the three experiments, every method warned about 74 hours (roughly three days) before the bearing failed and correctly named the damaged part (the outer ring); on the two other experiments, which were kept aside for a fair test, the system also warned in advance, but the methods that warn earliest also raise more false alarms, and a restart of the machine can fool the system.

Reading map.

- **If you have no technical background**, read this section, the Introduction (Section 1), every light-blue box titled *In plain words*, every grey box titled *How to read this figure*, the bold *What do we see?* / *What does it mean?* parts after each figure, the “Summary of findings” (Section 6.5) and the Conclusion (Section 10).
- **You can safely skim** the equations and the program code (the boxed listings in typewriter font). Each equation is followed by an explanation of every symbol, and each code listing has a plain explanation before and after it.
- **The Software engineering section** (Section 8) is mainly for readers who want to run the project themselves.
- **Unknown word?** The *Glossary of Key Terms* in Appendix A explains every technical term used in this report in one to three plain sentences.

In plain words

Boxes like this one appear in every technical section. They say, without formulas, what the section is about and why it is needed. You can read only these boxes and still follow the whole project.

How to read this figure

Boxes like this one appear right after every figure. They explain what the horizontal and vertical axes show (with units), what each colour, line and shaded area means, and any unusual scale.

1 Introduction

1.1 Problem

Predictive maintenance (repairing a machine based on its measured condition rather than on a fixed calendar) aims to repair a machine *shortly before* it fails: late enough to use most of a component’s life, early enough to plan the intervention. For rotating machines, vibration is the richest source of information, and bearings are the component that fails most often. A sensor-equipped machine streams vibration data continuously, but almost never comes with labels (notes saying what state the machine was in) such as “this bearing is now degrading”. Failures are rare, expensive and different from each other, so a model trained to classify known failures will not be available for a new machine.

The question this project answers is therefore:

Using only data from a period in which the machine is assumed healthy, can we warn an operator early, reliably and with few false alarms that a bearing is starting to fail, and say what kind of defect it probably is?

1.2 Approach in one paragraph

Every vibration snapshot (a one-second recording) is turned into a compact list of condition indicators (summary numbers that describe the state of the bearing). The first 25 % of each run is treated as healthy and used to fit an anomaly detector (a program that learns “normal” and scores how unusual new data is); the next 10 % sets an alarm threshold (the score above which we worry). From then on the detector scores every new snapshot, an alarm is raised when 4 of the last 6 scores exceed the threshold, and the defect frequency whose envelope energy has grown most names the likely failure mode. The system is evaluated on three run-to-failure experiments by the number of hours between the alarm and the end of the test (*lead time*) and by the number of alarms raised while the machine was still healthy (*false alarms*).

1.3 Contributions

- A leakage-free, time-ordered evaluation protocol for the IMS dataset, applied to all three tests and all failed bearings, with test 2 used for development and tests 1 and 3 held out.
- Physically grounded features including envelope analysis at the bearing defect frequencies, computed for the exact bearing geometry of the test rig.
- A comparison of a classic baseline, two classical machine-learning detectors and a deep-learning sequence model under identical splits, thresholds and alarm logic.
- Operator-facing alarm logic (debouncing, persistence, first-to-alarm localisation and defect diagnosis) and an interactive dashboard that replays a test as a live stream.
- A reproducible software package: configuration file, Makefile, unit tests, committed feature tables (3 MB) so that every result can be regenerated without downloading 6 GB of raw data.

In plain words

What is new here? (1) The system is graded fairly: it never sees the future while it learns, and two of the three experiments were kept aside as a final exam. (2) The summary numbers are based on the physics of bearings, so they can also tell *which part* is damaged. (3) Four different detection methods, from a very simple one to a modern neural network, are compared under exactly the same rules. (4) The alarms are designed for a human operator: they ignore single blips, point to the bearing that is most likely the source, and name the probable defect; a dashboard shows all this on screen. (5) Anyone can re-run the whole study with one command.

1.4 Relation to an earlier attempt

This project restarts an earlier notebook (2025) that trained an LSTM autoencoder on the same dataset. Reviewing it revealed several methodological problems that invalidate its results. They are worth listing because they are common with this dataset and motivated the design decisions in Section 5.8:

1. **The model was trained on failure data.** The training set was the *last* million samples of the run, i.e. the period in which the bearing was failing, and the same data was then used for testing.
2. **The wrong bearing was monitored.** Only the first channel of test 1 (bearing 1) was used, but in test 1 bearings 3 and 4 fail and bearing 1 survives.
3. **Time was lost.** All 2 156 files were concatenated into one signal and cut into windows of 50 samples (2.5 ms). Without a time axis the question “how many hours before failure did it warn?” cannot be answered.
4. **Leakage between training and validation.** Overlapping windows were shuffled before the train/validation split, so the validation loss was optimistic.

The idea of the earlier work (learn “normal” with an autoencoder and use reconstruction error as an anomaly score) is kept; it is the LSTM autoencoder detector in this report, now set up correctly.

In plain words

An earlier version of this project made four mistakes that made its results look better than they were. It learned from the period when the bearing was already breaking (like studying with the answer sheet), it watched a bearing that never failed, it threw away the clock so it could not say “how many hours in advance”, and it mixed up the practice data with the check data. This report keeps the good idea of the earlier work but fixes all four mistakes.

2 Background**2.1 Rolling-element bearings and their defects****In plain words**

A bearing looks like two metal rings, one inside the other, with a row of small balls or rollers between them. The inner ring turns with the shaft; the outer ring sits still in the machine’s housing; a light frame called the *cage* keeps the rollers evenly spaced. Over time a small chip or crack can form on one of these surfaces. Each time a roller rolls over the chip, it makes a tiny knock, like a car tyre hitting a pothole. Because the machine turns at a steady speed,

the knocks come at a steady rhythm, and that rhythm is different for each part of the bearing. So if we can hear the rhythm, we can tell *where* the damage is. The formulas below simply calculate these rhythms from the size of the bearing and the speed of the shaft.

A rolling-element bearing consists of an inner race (mounted on the shaft), an outer race (mounted in the housing), rolling elements (balls or rollers) and a cage that keeps the rolling elements apart. Fatigue, contamination or poor lubrication create a local defect (a spall, i.e. a small flake of metal breaking out of the surface, or a crack) on one of these surfaces. Every time a rolling element passes over the defect it produces a short mechanical impact. These impacts repeat at a rate that depends only on the bearing geometry and the shaft speed, and they excite the structural resonances of the bearing and housing (the natural “ringing tones” of the metal, like the note of a bell when it is struck), typically in the kHz range (thousands of vibrations per second).

For a shaft frequency f_r , n rolling elements of diameter d , pitch diameter D and contact angle ϕ , the characteristic defect frequencies are

$$\text{BPFO} = \frac{n}{2} f_r \left(1 - \frac{d}{D} \cos \phi \right) \quad (\text{ball pass frequency, outer race}) \quad (1)$$

$$\text{BPFI} = \frac{n}{2} f_r \left(1 + \frac{d}{D} \cos \phi \right) \quad (\text{ball pass frequency, inner race}) \quad (2)$$

$$\text{BSF} = \frac{D}{2d} f_r \left(1 - \left(\frac{d}{D} \cos \phi \right)^2 \right) \quad (\text{ball spin frequency, rolling element}) \quad (3)$$

$$\text{FTF} = \frac{f_r}{2} \left(1 - \frac{d}{D} \cos \phi \right) \quad (\text{fundamental train / cage frequency}) \quad (4)$$

What the symbols mean. f_r is how many full turns the shaft makes per second (in hertz, Hz). n is the number of rollers in one row. d is the diameter of one roller. D is the *pitch diameter*: the diameter of the imaginary circle that passes through the centres of all rollers. ϕ (the Greek letter “phi”) is the *contact angle*: how much the rollers are tilted where they touch the rings. The term $\frac{d}{D} \cos \phi$ is a small correction number that depends only on the shape of the bearing.

What the formulas do. BPFO counts how many rollers pass over one fixed point of the outer ring per second. Roughly half of the rollers’ count times the shaft speed, slightly reduced because the cage that carries the rollers turns more slowly than the shaft. BPFI is the same count for a point on the inner ring; it is slightly larger, because the inner ring turns towards the rollers. BSF is how fast each roller spins around its own axis, and FTF is how fast the cage goes round (a bit less than half the shaft speed). If damage is on the outer ring, we expect knocks at the BPFO rhythm; on the inner ring, at the BPFI rhythm; and so on.

The IMS test rig uses Rexnord ZA-2115 double-row bearings with $n = 16$ rollers per row, $d = 0.331$ in, $D = 2.815$ in and $\phi = 15.17^\circ$ at 2000 rpm (revolutions per minute; $f_r = 33.3$ Hz, i.e. 33.3 turns per second). Table 1 lists the resulting frequencies; the implementation is shown in Listing 1.

Table 1: Characteristic defect frequencies of the ZA-2115 bearing at 2000 rpm.

Frequency	Value [Hz]	Defect it indicates
BPFO	236.4	outer race
BPFI	296.9	inner race
BSF	139.9	rolling element
FTF	14.8	cage

How to read Table 1. Each row is one “rhythm”. For example, a damaged spot on the outer

ring should produce about 236 knocks per second; a damaged spot on the inner ring about 297 knocks per second. These numbers are the “fingerprints” we will look for later.

What the code below does. It turns the formulas above into a small piece of Python so that the computer calculates the four rhythms from the measured bearing dimensions, instead of us typing the numbers by hand.

Listing 1: Defect frequencies from the bearing geometry (`src/bearing_efw/dataset.py`).

```

1 FS = 20_480 # sampling rate [Hz]
2 SHAFT_HZ = 2000 / 60.0
3 N_ROLLERS, ROLLER_D, PITCH_D, CONTACT_ANGLE_DEG = 16, 0.331, 2.815, 15.17
4
5 def fault_frequencies(fr: float = SHAFT_HZ) -> dict[str, float]:
6     """Characteristic defect frequencies [Hz] for the ZA-2115 bearing."""
7     ratio = ROLLER_D / PITCH_D * np.cos(np.deg2rad(CONTACT_ANGLE_DEG))
8     return {
9         "bpfo": N_ROLLERS / 2 * fr * (1 - ratio), # outer race
10        "bpfi": N_ROLLERS / 2 * fr * (1 + ratio), # inner race
11        "bsf": PITCH_D / (2 * ROLLER_D) * fr * (1 - ratio**2), # rolling element
12        "ftf": fr / 2 * (1 - ratio), # cage
13    }

```

Line by line. Line 1 stores the sampling rate: the sensor takes 20 480 measurements every second. Line 2 converts the shaft speed from turns per minute (2000) to turns per second. Line 3 stores the bearing’s dimensions. Line 5 starts a function (a reusable recipe) called `fault_frequencies`. Line 7 computes the small correction number $\frac{d}{D} \cos \phi$ once (the angle is first converted from degrees to radians, the unit the computer expects). Lines 8–13 return the four rhythms, one per line, written exactly as in the equations above.

2.2 Vibration condition indicators

In plain words

One second of vibration is 20 480 numbers: far too many to compare directly. So each recording is summarised into a handful of meaningful numbers, much like a blood test summarises your health into a few values (sugar, cholesterol, ...). We use three families of summary numbers: (1) how *strong* and how *spiky* the shaking is; (2) *in which pitch range* (low, middle, high) the shaking happens; (3) whether the shaking contains a *regular knocking rhythm* that matches one of the bearing’s fingerprints from Table 1.

Time-domain statistics. For a zero-mean snapshot $x_1, \dots, x_N$:

$$\text{RMS} = \sqrt{\frac{1}{N} \sum_i x_i^2}, \quad \text{kurtosis} = \frac{\frac{1}{N} \sum_i x_i^4}{\text{RMS}^4}, \quad \text{crest factor} = \frac{\max_i |x_i|}{\text{RMS}}. \quad (5)$$

What the symbols mean. x_i is the i -th measured value in the recording (how strongly the sensor was shaken at that instant, in units of g , the acceleration of gravity). N is the number of values (20 480). “Zero-mean” means the average value has been subtracted, so the signal swings around zero. $\sum_i$ means “add up over all values”, $\max_i |x_i|$ is the single largest swing (ignoring its sign), and $\sqrt{}$ is the square root.

What the formulas do. *RMS* (root mean square) squares every value (so that swings downwards count as much as swings upwards), averages them, and takes the square root. The result is the “typical size” of the shaking: the average strength of the vibration. It grows when the machine vibrates more. *Kurtosis* uses the fourth power, which makes large values count enormously more than small ones, and then divides by RMS^4 so that the result does not depend on how loud

the signal is. It measures *spikiness*: a smooth, random hum gives about 3, while a signal with occasional sharp knocks gives a much larger value. *Crest factor* divides the largest single swing by the typical size; a tall spike among calm values gives a high crest factor.

RMS measures overall vibration energy and is the indicator used by most industrial standards (e.g. ISO 10816/20816). Kurtosis equals 3 for Gaussian noise (purely random, “hiss-like” vibration) and grows when the signal contains sharp impacts, which makes it sensitive to early, localized defects even before the energy rises. Crest, shape ($\text{RMS}/\overline{|x|}$, where $\overline{|x|}$ is the average size of the swings) and impulse ($\max |x|/\overline{|x|}$) factors are related impulsiveness measures (other ways of measuring spikiness); skewness captures asymmetry (whether the signal swings further on one side of zero than on the other).

Spectral band energies. The fraction of the power spectrum (a breakdown of the signal by pitch, like the bars of a music equaliser) in six bands (0–1, 1–2, 2–4, 4–6, 6–8 and 8–10.24 kHz) and the spectral centroid (the “average pitch” of the vibration, weighted by strength) describe *where* the energy sits. A developing defect shifts energy towards the high-frequency resonances.

Envelope analysis.

In plain words

Imagine someone knocking on a door in a noisy room. You may not make out the sound of each knock, but you can still hear the *rhythm* of the knocking. Envelope analysis does the same with vibration: it first keeps only the high-pitched “ringing” that each knock causes, then traces the outline of how loud that ringing is from moment to moment (the *envelope*), and finally asks how often that outline repeats per second. If it repeats 236 times per second, the knocks come from the outer ring (Table 1).

Defect impacts are short and excite high-frequency resonances; their *repetition rate* (e.g. BPFO) is invisible in the raw spectrum because it is hidden under the resonance. Envelope analysis recovers it in three steps [3]:

1. band-pass filter the signal around the resonance region (here 2–8 kHz, 4th-order Butterworth, zero-phase). In plain terms: keep only the pitches between 2 000 and 8 000 vibrations per second and throw the rest away, like tuning a radio to one station. The filter is a standard smooth design (Butterworth) and is run forwards and backwards (zero-phase) so it does not shift the signal in time;
2. take the magnitude of the analytic signal (Hilbert transform), which is the envelope of the amplitude-modulated resonance. In plain terms: the Hilbert transform is a mathematical tool that draws a smooth curve along the tops of the fast oscillations, giving the “loudness outline” of the ringing;
3. compute the spectrum of the envelope; a localized defect produces a peak at its characteristic frequency and its harmonics (whole-number multiples of it: $1\times$, $2\times$, $3\times$, ...).

The feature used here is the share of envelope-spectrum energy (5–1000 Hz) that lies within ± 3 Hz of the first three harmonics of BPFO, BPF1 and BSF. For the outer race, for example, this is the fraction of all rhythm energy that sits close to 236, 473 or 709 knocks per second. A healthy bearing gives a small fraction; an outer-race defect makes it grow.

2.3 Unsupervised anomaly detection

In plain words

“Unsupervised” means that nobody tells the computer which recordings are healthy and which are faulty. We only assume that the beginning of the run is healthy. Each detector learns what healthy looks like and then gives every new recording an *anomaly score*: a number that is small for familiar, healthy-looking data and large for unfamiliar data. Think of a security guard who knows every regular face in an office building: they do not need a list of criminals to notice a stranger. We use four “guards” with different styles, described below.

Because no failure labels are used, all detectors are *one-class* or *novelty* detectors: they are fitted on healthy data only and assign a score that grows as a new observation becomes less similar to it.

Mahalanobis distance. With mean μ and covariance Σ of the healthy feature vectors,

$$d_M(\mathbf{x}) = \sqrt{(\mathbf{x} - \mu)^\top \Sigma^{-1} (\mathbf{x} - \mu)}. \quad (6)$$

What the symbols mean. $\mathbf{x}$ is the list of summary numbers (features) of a new recording. μ (“mu”) is the average list over the healthy period. Σ (“Sigma”) is the *covariance*: a table that records how much each feature normally varies and which features normally rise and fall together. Σ^{-1} is its inverse (it plays the role of “dividing by” the normal variation), and $^\top$ (transpose) is a bookkeeping step that makes the whole product come out as one single number.

What the formula does. It measures how far the new recording is from the healthy average, counted in units of “normal variation” and taking into account which features usually move together. For example, RMS and peak normally rise together; if both rise a little, that is not surprising, but if one rises while the other falls, that is unusual and the distance becomes large.

It accounts for correlations between features (e.g. RMS and peak). Because many features are correlated and the training set is small, Σ is estimated with Ledoit–Wolf shrinkage [5], which pulls the sample covariance towards a scaled identity and keeps it well conditioned. In plain terms: with few examples, the estimate of “which features move together” is shaky, so Ledoit–Wolf blends it with a simpler, safer guess (“each feature varies on its own”), much like averaging your own uncertain estimate with a sensible default.

Isolation Forest. An ensemble of random trees that isolate observations by random splits [4]. Anomalies are few and different, so they are isolated in fewer splits; the score is derived from the average path length. In plain terms: imagine a game of “twenty questions” with random yes/no questions such as “is the RMS larger than 0.4?”. An ordinary recording hides among many similar ones and needs many questions to be singled out; an odd one is singled out after only a few. Isolation Forest plays this game hundreds of times (“trees”) and uses the average number of questions as the score: fewer questions means more unusual.

LSTM autoencoder. An autoencoder compresses its input into a low-dimensional latent code and reconstructs it. Trained only on healthy data, it reconstructs healthy inputs well and unfamiliar inputs poorly, so the reconstruction error

$$e_t = \frac{1}{Wp} \sum_{k=0}^{W-1} \sum_{j=1}^p (x_{t-k,j} - \hat{x}_{t-k,j})^2 \quad (7)$$

over a window of W snapshots and p features is an anomaly score [7]. Long short-term memory (LSTM) layers [6] let the model use the temporal evolution inside the window.

What the symbols mean. e_t is the error (the anomaly score) at snapshot number t . W is the number of consecutive snapshots the model looks at together (the *window*), and p is the number of features per snapshot. $x_{t-k,j}$ is the real value of feature j in the snapshot k steps before t , and $\hat{x}_{t-k,j}$ (“x-hat”) is the model’s rebuilt version of the same value. The two $\sum$ signs add up over all snapshots in the window and all features, and $\frac{1}{Wp}$ turns the sum into an average.

What the formula does. It is the average squared mistake of the rebuilt copy. Think of a person who has only ever seen healthy patterns and is asked to redraw a pattern from a short summary: they redraw healthy patterns almost perfectly but make big mistakes with patterns they have never seen. A big mistake therefore means “this does not look healthy”. An LSTM (long short-term memory) is a kind of neural network that reads data in order and remembers what came before, so it can notice changes in how the features evolve over a couple of hours, not just in one snapshot.

RMS threshold. The baseline every maintenance engineer would try first: an alarm when the overall vibration level rises above its healthy range. Any more complex method must beat it to be worth its complexity.

3 Dataset

In plain words

We did not collect the data ourselves. Researchers at the University of Cincinnati built a test machine with four bearings on one shaft, pressed the shaft down with a heavy load, and let it run day and night until a bearing broke. Every 10 minutes, sensors on the bearings recorded one second of vibration. They repeated this experiment three times (“test 1”, “test 2”, “test 3”). For each test we know which bearing was broken at the end and how, but *not* when the damage started: that is exactly what our system has to find out. Before use, the data needed some cleaning (unusual file packaging, empty recordings after shut-down, and long pauses in the recording).

The IMS bearing dataset [1, 2] was produced by the NSF I/UCR Center for Intelligent Maintenance Systems at the University of Cincinnati and is distributed by the NASA Prognostics Center of Excellence. Four bearings are mounted on one shaft driven at a constant 2000 rpm by an AC motor, with a 6000 lb (about 2.7 tonnes) radial load applied by a spring mechanism. Accelerometers (PCB 353B33; sensors that measure shaking) on the bearing housings are sampled at 20 kHz (20 000 measurements per second) by an NI DAQCard-6062E (a data-recording card). Every 10 minutes (5 minutes at the start of test 1) a one-second snapshot of 20 480 samples per channel is written to a text file named after its timestamp. Each test ran until a bearing failed (hence “run-to-failure”).

Table 2: The three run-to-failure tests. Duration is measured after dropping dead snapshots.

Test	Snapshots	Channels	Duration [h]	Failure at end of test
test1	2 156	8 (x, y per bearing)	827.6	bearing 3 inner race, bearing 4 roller element
test2	984	4	163.5	bearing 1 outer race
test3	6 324	4	1 073.1	bearing 3 outer race

How to read Table 2. “Snapshots” is the number of one-second recordings. “Channels” is the number of sensors: test 1 had two sensors per bearing (one measuring up-and-down, one side-to-side shaking), the other tests one per bearing. “Duration” is how long the machine ran, in

hours: test 2 ran for about 7 days, test 3 for about 44 days. The last column says what was found broken when the test was stopped.

Practical quirks discovered while working with the data.

- The archive is a ZIP containing a 7-Zip file containing three RAR files (about 1 GB download, 6.2 GB extracted, 9 464 snapshot files). (Three kinds of compressed “packages” nested inside each other, like Russian dolls.)
- The file `3rd_test.rar` extracts to a folder called `4th_test/txt`.
- Files are tab-separated with Windows line endings and no header.
- The last snapshot of test 2 and test 3 is a near-zero signal ($\text{RMS} \approx 0.001 \text{ g}$) recorded after shutdown; such “dead” snapshots are removed ($\text{RMS} < 0.01 \text{ g}$ on any channel).
- Test 1 was recorded in bursts with gaps of up to 148 hours; test 3 has stops of 4.5, 7.5 and 7.3 hours (at about hours 262, 354 and 1002). Test 2 is continuous.

Downloading. `scripts/download_data.py` downloads the archive from the PHM Society mirror of the NASA repository and unpacks all layers (it needs `py7zr` and the `unar` or `unrar` command). The raw data is excluded from Git (the version-control system, see Section 8.5); the extracted features are committed.

4 Tools and environment

In plain words

This section lists the software “toolbox”. The project is written in the Python programming language and uses well-known free libraries (ready-made building blocks) for maths, signal processing, machine learning, plotting and the dashboard. You do not need to know these tools to understand the results. The important message is at the end: everything runs on an ordinary laptop in a few minutes, without special hardware.

Table 3: Software used in the project.

Tool	Role
Python 3.10+	implementation language
NumPy, pandas	numerical arrays, feature tables, time handling
SciPy (<code>signal</code> , <code>stats</code>)	Butterworth filtering, Hilbert transform, kurtosis/skewness
scikit-learn	Ledoit–Wolf covariance, Isolation Forest
PyTorch	LSTM autoencoder (CPU is sufficient)
Matplotlib	static figures for the README and this report
Streamlit, Plotly	interactive replay dashboard
PyYAML	experiment configuration (<code>configs/default.yaml</code>)
pytest	unit tests and dashboard smoke tests (Streamlit <code>AppTest</code>)
Jupyter / nbconvert	executed walkthrough notebook
py7zr, unar	extracting the nested 7z/RAR archive
GNU Make	one-command entry points (<code>make experiment</code> , <code>make dashboard</code> , ...)
Git, GitHub	version control, pull requests
L ^A T _E X (pdfLaTeX, TikZ, listings)	this report

The whole pipeline runs on a laptop CPU (the ordinary processor, no graphics card needed): feature extraction of 9 464 files takes about 1.5 minutes with 4 processes, and fitting and evaluating all four detectors on all three tests about 1.5 minutes.

5 Methodology

In plain words

This section explains, step by step, how a raw recording becomes an alarm. Think of an assembly line with five stations: (1) take the one-second recording; (2) summarise it into 18 numbers; (3) express those numbers relative to the bearing’s own normal; (4) let a detector turn them into one “how unusual” score; (5) decide whether to raise an alarm and, if so, name the probable defect.

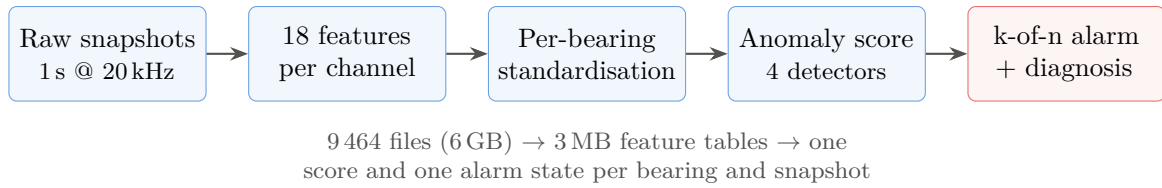


Figure 1: Processing pipeline.

How to read this figure

Read the boxes from left to right. Each box is one processing step, and each arrow means “the result of this step is handed to the next one”. The last box is drawn in red because it is where alarms are produced. The grey line underneath shows how much the data shrinks along the way: from 6 GB of raw recordings to a 3 MB table of summary numbers, and finally to one score and one alarm state (on or off) per bearing and per recording.

5.1 Feature extraction

In plain words

Here every recording from every sensor is turned into 18 summary numbers (“features”): 8 that describe strength and spikiness, 6 that say how the energy is shared between low and high pitches, 1 “average pitch”, and 3 that measure the knocking rhythm at the outer-ring, inner-ring and roller fingerprints. Doing this for nearly 10 000 files takes time, so the work is shared between several processor cores, like opening several checkout lanes in a supermarket.

Each snapshot and channel is reduced to 18 features (Listing 2): 8 time-domain statistics, 6 relative band energies, the spectral centroid and 3 envelope energies. Test 1 has two channels per bearing, so each test 1 bearing is described by 36 features; tests 2 and 3 by 18.

What the code below does. It computes the “pitch” features and the “knocking rhythm” features for one recording, following the three envelope steps described in the Background section.

Listing 2: Spectral and envelope features (excerpt of `src/bearing_efw/features.py`).

```

1 BANDS = [(0, 1000), (1000, 2000), (2000, 4000), (4000, 6000), (6000, 8000), (8000, 10240)]
2 ENVELOPE_BAND = (2000, 8000)
3 FAULT_TOL_HZ = 3.0 # half-width of the window around each defect frequency
4 N_HARMONICS = 3
5 _SOS = signal.butter(4, ENVELOPE_BAND, btype="bandpass", fs=FS, output="sos")

```

```

6
7 def spectral_features(x: np.ndarray) -> dict[str, float]:
8     x = x - x.mean()
9     freqs = np.fft.rfftfreq(len(x), 1 / FS)
10    psd = np.abs(np.fft.rfft(x * np.hanning(len(x)))) ** 2 / len(x)
11    total = psd.sum()
12    out = {f"band_{lo // 1000}_{hi // 1000}k": _band_energy(freqs, psd, lo, hi) / total
13           for lo, hi in BANDS}
14    out["spectral_centroid"] = float((freqs * psd).sum() / total)
15
16    # Envelope spectrum: impacts from a localized defect modulate the resonance
17    # at the defect frequency, which shows up after demodulation.
18    env = np.abs(signal.hilbert(signal.sosfiltfilt(_SOS, x)))
19    env -= env.mean()
20    env_spec = np.abs(np.fft.rfft(env * np.hanning(len(env)))) / len(env)
21    env_total = env_spec[(freqs > 5) & (freqs < 1000)].sum()
22    for name in ("bpfo", "bpfi", "bsf"):
23        e = sum(_band_energy(freqs, env_spec,
24                             k * _FAULT_F[name] - FAULT_TOL_HZ, k * _FAULT_F[name] +
25                             FAULT_TOL_HZ)
26               for k in range(1, N_HARMONICS + 1))
27    out[f"env_{name}"] = e / env_total
28    return out

```

Line by line. Lines 1–4 are fixed settings: the six pitch bands (in Hz), the 2–8 kHz band used for the envelope, the ± 3 Hz tolerance around each fingerprint, and the number of harmonics (3). Line 5 designs the band-pass filter once, so it can be reused for every file. Line 8 removes the average so the signal swings around zero. Line 9 lists which pitch each point of the spectrum stands for. Line 10 computes the spectrum with the FFT (fast Fourier transform, the standard method for splitting a signal into its pitches); multiplying by a Hanning window first gently fades the recording in and out, which avoids fake pitches caused by the abrupt start and end. Line 11 adds up the total energy. Lines 12–13 compute the share of energy in each of the six bands, and line 14 the average pitch. Line 18 is the whole envelope recipe in one line: filter (`sosfiltfilt`), Hilbert transform, and magnitude (`np.abs`). Lines 19–21 compute the spectrum of the envelope and its total energy between 5 and 1000 Hz. Lines 22–26 go through the three defect types and add up the envelope energy in narrow windows around $1\times$, $2\times$ and $3\times$ their fingerprint, divided by the total: the result is the *share* of rhythm energy that matches that defect.

Relative (rather than absolute) band and envelope energies are used so that these features describe the *shape* of the spectrum and are not simply copies of RMS. Feature extraction is parallelised over files with a process pool (Listing 3); the result is one compressed CSV (a simple spreadsheet-like text table) per test.

What the code below does. It reads every recording, computes the features for each bearing and sensor, and spreads this work over several processor cores so it finishes faster.

Listing 3: Parallel feature extraction (excerpt of `scripts/build_features.py`).

```

1 def _process(args: tuple[str, Path]) -> list[dict]:
2     test, path = args
3     spec = TESTS[test]
4     data = load_snapshot(path)          # (20480, n_channels) float32
5     ts = parse_timestamp(path.name)    # file name = acquisition time
6     rows = []
7     for bearing, cols in spec.channels.items():
8         for axis, col in enumerate(cols):
9             rows.append({"timestamp": ts, "bearing": bearing, "channel": axis,
10                         **snapshot_features(data[:, col])})
11     return rows
12
13 with ProcessPoolExecutor(workers) as ex:

```

```

14 rows = [r for chunk in ex.map(_process, [(test, f) for f in files], chunksize=16)
15         for r in chunk]

```

Line by line. Lines 1–11 handle a single file. Line 4 loads the recording (20 480 rows, one column per sensor), and line 5 reads the recording time from the file name. Lines 7–10 loop over every bearing and every sensor of that bearing and store one row containing the time, the bearing number, the sensor number and all 18 features. Lines 13–15 hand the list of all files to a pool of worker processes (`ProcessPoolExecutor`), which process files in parallel, 16 at a time per worker, and collect all rows into one table.

5.2 Signal-level illustration

In plain words

Before trusting any automatic method, we look at the raw data with our own eyes. We take the bearing that failed in test 2 and compare three recordings, like three photos taken at different moments: when it was healthy (hour 0), when it was already damaged (hour 100), and shortly before it broke (hour 160). The three figures below show what the eye can and cannot see, and why the special “rhythm” feature is needed.

Figures 2–4 show what the features capture on the failing bearing of test 2 at three moments: healthy (hour 0), degraded (hour 100, after the alarm) and near failure (hour 160).

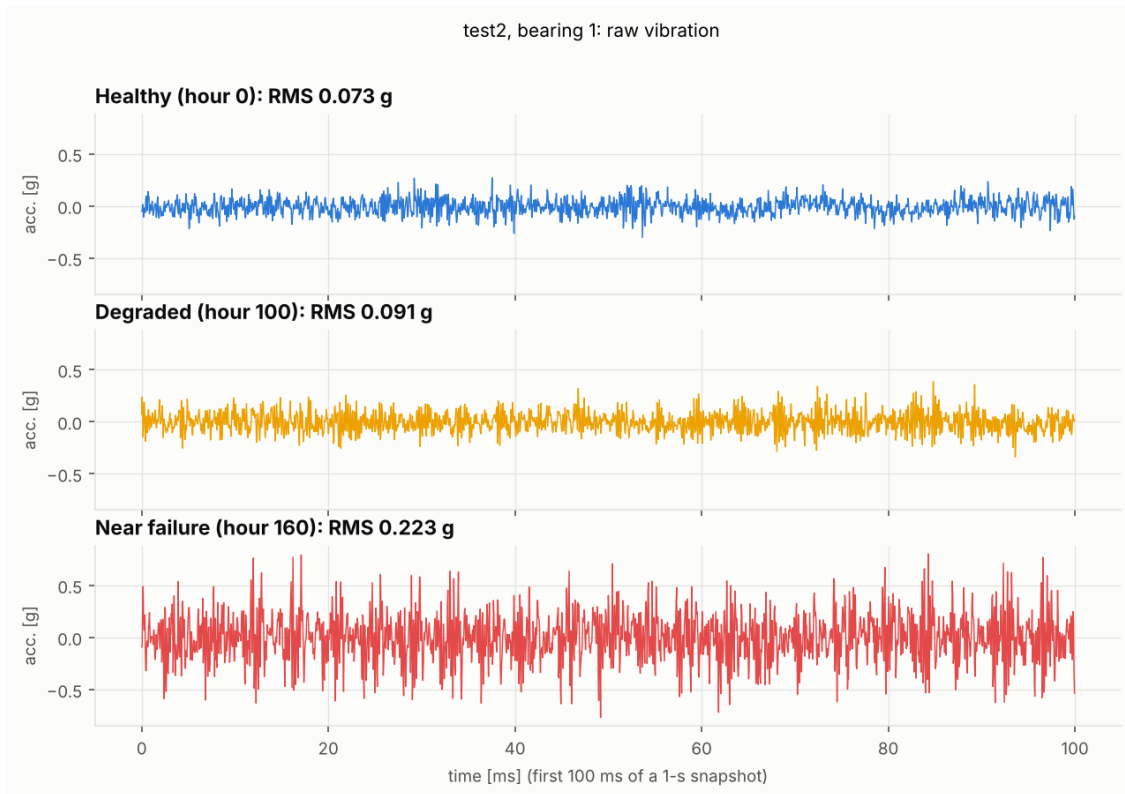


Figure 2: Raw vibration of test 2, bearing 1 (first 100 ms of three snapshots).

How to read this figure

There are three panels stacked on top of each other, one per moment: **blue** = healthy (hour 0), **orange** = degraded (hour 100), **red** = near failure (hour 160). The horizontal axis is time in milliseconds (ms; 1 000 ms = 1 second); only the first 100 ms, i.e. one tenth of each one-second

recording, is shown. The vertical axis is the acceleration measured by the sensor, in g ($1g$ is the acceleration of gravity). Values go above and below zero because the sensor shakes back and forth. All three panels use the same vertical scale, so heights can be compared directly. The RMS value (typical vibration strength) of each recording is written in its title.

What do we see? At hour 100 the waveform is still visually similar to the healthy one and RMS has risen only from 0.073 to 0.091 g (+25 %). Near failure the amplitude has tripled (0.223 g) and the signal is clearly impulsive (full of sharp spikes).

What does it mean? A rise of 25 % is an amount that would be easy to miss by eye or with a fixed vibration limit, even though the bearing is already damaged at hour 100 (as Figure 4 will show). An early warning must therefore rely on more than visual inspection of the waveform or the overall level.

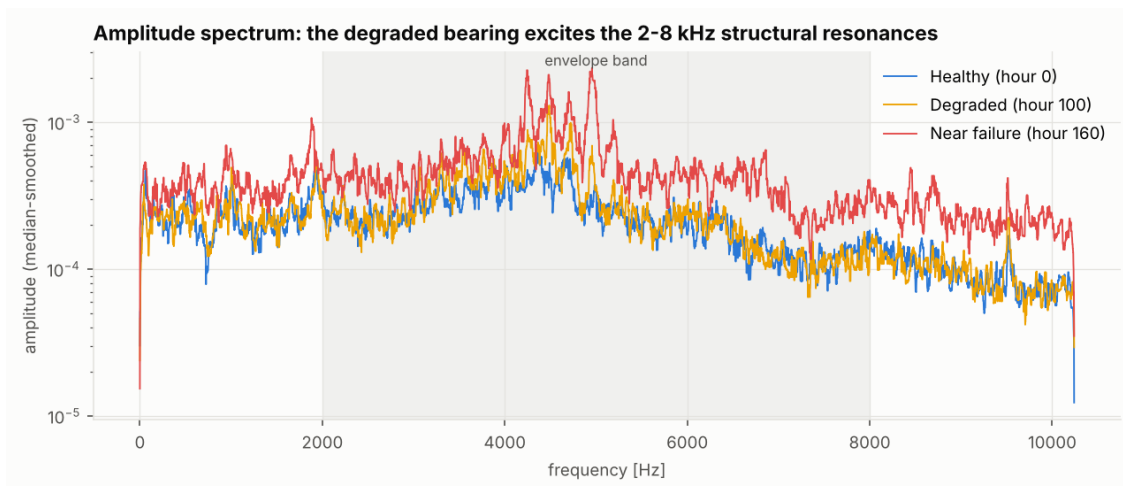


Figure 3: Amplitude spectrum of the same three snapshots (median-smoothed, log scale).

How to read this figure

This figure shows the same three recordings split into pitches (a *spectrum*). The horizontal axis is frequency in hertz (Hz, vibrations per second), from 0 to about 10 000 Hz (the highest pitch that 20 480 measurements per second can capture). The vertical axis is the strength of each pitch. It uses a **log scale**: each grid line is ten times larger than the one below it (10^{-5} , 10^{-4} , 10^{-3}), which lets very small and very large values fit in the same picture. “Median-smoothed” means each point was replaced by the middle value of its neighbours to make the lines less jagged. Colours as before: **blue** healthy, **orange** degraded, **red** near failure. The **grey shaded area** from 2 000 to 8 000 Hz, labelled “envelope band”, is the pitch range used later for envelope analysis.

What do we see? The energy of all three spectra is concentrated between roughly 3 and 6 kHz. As the bearing degrades, the peaks in this range grow and broaden, and near failure the high-frequency floor (6–10 kHz) rises by almost an order of magnitude (almost ten times).

What does it mean? The 3–6 kHz peaks are the structural resonances of the bearing housing: the “ringing tones” of the metal that each knock sets off. The growth of these peaks justifies both the relative band-energy features and the choice of the 2–8 kHz band for envelope analysis.

Why does it matter? The spectrum tells us *that* something is changing, but it does not show *which* component is damaged. For that we need Figure 4.

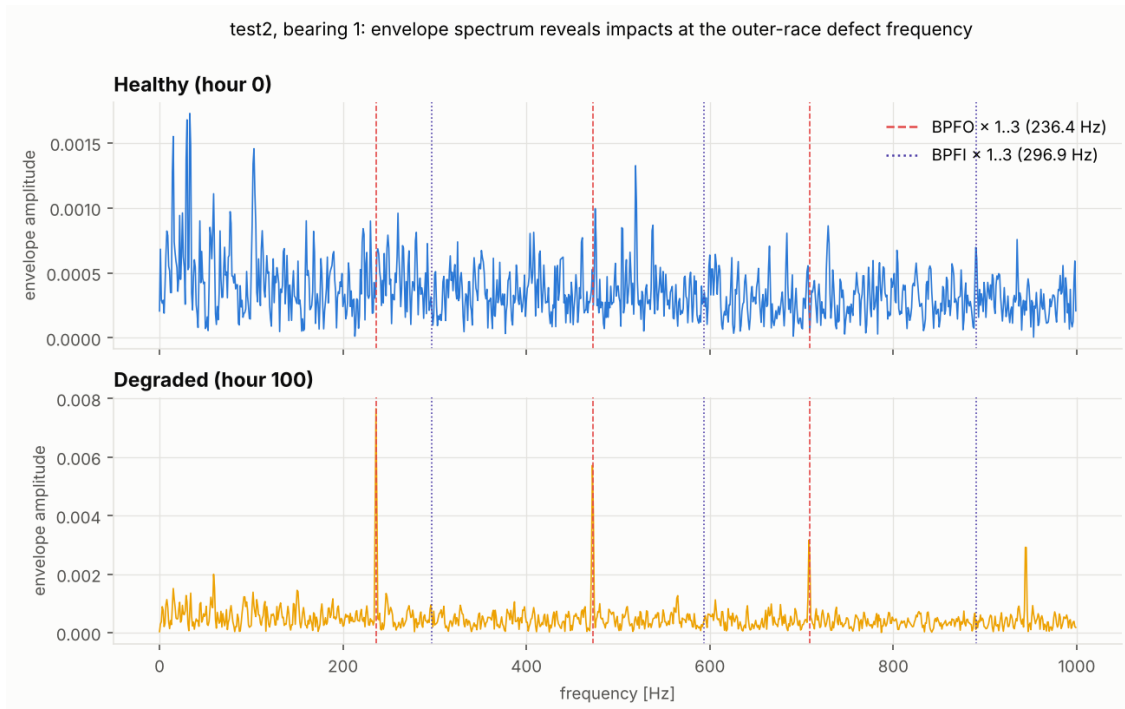


Figure 4: Envelope spectrum of the healthy and degraded snapshots, with the first three harmonics of BPFO (dashed) and BPFI (dotted).

How to read this figure

Two panels: top = healthy recording (**blue**, hour 0), bottom = degraded recording (**orange**, hour 100). The horizontal axis is the *rhythm* frequency in Hz: how many times per second the ringing gets louder and softer, i.e. how many knocks per second. The vertical axis is the strength of each rhythm (“envelope amplitude”, a relative quantity without a physical unit). Note that the two panels have **different vertical scales**: the top one goes up to about 0.0015, the bottom one up to 0.008. The **red dashed vertical lines** mark the outer-ring fingerprint and its multiples (236, 473 and 709 Hz); the **purple dotted vertical lines** mark the inner-ring fingerprint and its multiples (297, 594 and 891 Hz). A tall peak sitting exactly on the red lines points to an outer-ring defect; on the purple lines, to an inner-ring defect.

What do we see? This is the key diagnostic figure. The healthy envelope spectrum is a flat noise floor (random small bumps) with no structure at the defect frequencies. At hour 100 sharp peaks appear exactly at 236, 473 and 709 Hz, i.e. at $1\times$, $2\times$ and $3\times$ BPFO, with amplitudes about ten times the noise floor, while nothing appears at the inner-race frequencies.

What does it mean? This is the textbook signature of a localized **outer-race** defect: a damaged spot on the outer ring is being hit about 236 times per second. It confirms that the bearing geometry, the defect-frequency calculation and the envelope pipeline are correct, and it is what the `env_bpfo` feature measures.

Why does it matter? The defect is clearly visible here about three days before the end of the test, at a moment when the raw waveform (Figure 2) still looked almost normal. This is why envelope features make early warning *and* diagnosis possible.

5.3 Preprocessing and data splits

In plain words

Two things happen here. First, each bearing’s life is cut into three periods in time order, like a new employee’s first months: the first 25 % is the *training* period (the system learns what normal looks like), the next 10 % is the *calibration* period (the system decides how high a score must be before it is worrying), and the rest is the *test* period (the system is on duty). Second, every feature is expressed relative to that bearing’s own normal: “how many normal wiggles above or below its usual value”. This is like judging a person’s temperature against their own usual temperature rather than against everyone else’s: a naturally “loud” bearing is not mistaken for a sick one.

Features are pivoted (rearranged) into one matrix (table) per bearing (one row per snapshot, ordered in time). Strictly positive, heavy-tailed features (features that are usually small but occasionally very large) are log-transformed (replaced by their logarithm, which compresses large values so that “ten times larger” becomes one step rather than a huge leap), then every feature is standardized with the mean and standard deviation of that bearing’s own training period (Listing 4). Standardizing means: subtract the healthy average and divide by the healthy standard deviation (the typical size of normal fluctuations); the result is called a *z-score*. After this step a value of +3 means “three healthy standard deviations above this bearing’s normal”, regardless of whether the bearing is naturally loud or quiet, so one detector can serve all bearings of a test.

What the code below does. It cuts each bearing’s recordings into training and calibration periods, takes logarithms where needed, and converts every feature into a z-score using only the training period.

Listing 4: Time-ordered splits and per-bearing standardisation (excerpt of `src/bearing_efw/preprocess.py`).

```

1 for bearing, w in wide_by_bearing(df).items():
2     hours = ((w.index - t0).total_seconds() / 3600).to_numpy()
3     # Splits use the snapshot index, not wall-clock time: test1 was recorded in
4     # bursts with multi-day gaps, so a time-based split would leave almost no
5     # training data.
6     life = np.arange(len(w)) / len(w)
7     train = life < train_end                                # first 25 %
8     calib = (life >= train_end) & (life < calib_end)         # next 10 %
9
10    Z = w.copy()
11    log_cols = [c for c in Z.columns if c.rsplit("_ch", 1)[0] in LOG_FEATURES]
12    Z[log_cols] = np.log(Z[log_cols].clip(lower=1e-9))
13    mu, sd = Z[train].mean(), Z[train].std().replace(0, 1)
14    X = ((Z - mu) / sd).to_numpy(dtype=np.float32)

```

Line by line. Line 1 repeats everything for each bearing. Line 2 converts the recording times into “hours since the start of the test”. Line 6 gives each recording its position in the bearing’s life as a fraction between 0 and 1, by *counting recordings* rather than by looking at the clock (lines 3–5 explain why: test 1 has multi-day pauses). Lines 7–8 mark the first 25 % as training and the next 10 % as calibration. Lines 11–12 take the logarithm of the heavy-tailed features; `clip(lower=1e-9)` replaces exact zeros by a tiny number, because the logarithm of zero is undefined. Line 13 computes the healthy average (`mu`) and spread (`sd`) *from the training rows only*, so nothing from the future leaks in; a spread of zero is replaced by 1 to avoid dividing by zero. Line 14 produces the z-scores.

Table 4: Size of the splits per bearing.

Test	Snapshots	Train	Calibration	Train ends [h]	Calibration ends [h]
test1	2 156	539	216	290.1	433.8
test2	982	246	98	40.8	57.2
test3	6 323	1 581	633	267.5	380.2

How to read Table 4. Take test 2: of its 982 recordings, the first 246 (up to hour 40.8) are used to learn “normal”, the next 98 (up to hour 57.2) to set the alarm level, and everything after hour 57.2 is watched by the finished system.

5.4 Detectors

In plain words

These are the four “guards” introduced in the Background section. To keep the comparison fair, all four do exactly two things in the same way: *fit* (study the healthy data) and *score* (give each new recording one number; larger means stranger). Because every bearing has already been put on its own “normal = 0” scale, the healthy data of all four bearings can be pooled together, which gives the guards four times more examples to learn from.

All detectors share one interface: *fit* receives the list of healthy training matrices of the four bearings, and *score* maps a bearing’s full matrix to one score per snapshot. Fitting on the pooled, standardized healthy data of all bearings quadruples the training set compared with fitting one model per bearing.

What the code below does. It defines the three simpler detectors, each with its *fit* and *score* step.

Listing 5: Classical detectors (excerpt of `src/bearing_efw/models.py`).

```

1 class RMSThreshold:
2     """Classic condition-monitoring baseline: overall vibration level (z-score of log RMS)."""
3     def fit(self, Xs): return self
4     def score(self, X): return X[:, self.idx].max(axis=1) # idx = RMS columns
5
6 class Mahalanobis:
7     """Distance from the healthy feature distribution (shrinkage covariance)."""
8     def fit(self, Xs):
9         self.cov = LedoitWolf().fit(np.vstack(Xs))
10        return self
11    def score(self, X):
12        return np.sqrt(self.cov.mahalanobis(X))
13
14 class IForest:
15     def __init__(self, n_estimators=300, seed=0):
16         self.model = IsolationForest(n_estimators=n_estimators, random_state=seed)
17     def fit(self, Xs):
18         self.model.fit(np.vstack(Xs))
19         return self
20     def score(self, X):
21         return -self.model.score_samples(X) # higher = more anomalous

```

Line by line. *RMS threshold* (lines 1–4): there is nothing to learn (line 3), because the RMS has already been converted to a z-score; the score is simply that z-score (line 4). In test 1, where each bearing has two sensors, the larger of the two is used (`max`). *Mahalanobis* (lines 6–12): line 9 stacks the healthy data of all bearings (`np.vstack`) and estimates the covariance with Ledoit–Wolf;

line 12 returns the distance (the library gives the squared distance, so the square root is taken). *Isolation Forest* (lines 14–21): line 16 creates a forest of 300 random trees with a fixed random seed (so the result is the same every run); line 21 flips the sign of the library’s score so that, like the others, a higher score means “more unusual”.

LSTM autoencoder architecture. The encoder LSTM (32 hidden units) reads a window of $W = 12$ consecutive snapshots (about two hours); its last hidden state is projected to an 8-dimensional latent code. The decoder repeats the code over 12 steps, passes it through a second LSTM (32 units) and a linear layer back to the feature dimension. The model has 16 250 parameters for 18 features (19 148 for test 1’s 36 features). It is trained for 60 epochs with Adam (learning rate 2×10^{-3} , batch size 64) as a *denoising* autoencoder: Gaussian noise with standard deviation 0.1 is added to the input, and the model must reconstruct the clean window. On a training set of only about 1000 windows (test 2) this prevents the network from memorizing the training data. Windows are built per bearing, never across two bearings, and the window for snapshot t ends at t , so the score only uses past data and can run online.

In plain words

The neural network looks at the last 12 recordings (about two hours) of one bearing at once. The first half (the *encoder*) squeezes these 12×18 numbers into a summary of just 8 numbers; the second half (the *decoder*) tries to rebuild the original 12×18 numbers from that summary. The network has about 16 000 adjustable “knobs” (parameters). Training means turning these knobs, a little at a time, so the rebuilt copy matches the original. One *epoch* is one full pass over all training examples; we do 60 passes. *Adam* is the standard recipe for turning the knobs, the *learning rate* is how big each turn is, and the *batch size* (64) is how many examples are looked at before each turn. During training a little random noise is added to the input, and the network must still rebuild the clean version (a *denoising* autoencoder). This is like learning to recognise faces from slightly blurry photos: it forces the network to learn the general pattern instead of memorising each example. Finally, the score for a given moment only uses recordings up to that moment, so the system could run live on a real machine (“online”).

What the code below does. It defines the network (encoder, 8-number summary, decoder), the training loop with the noise trick, and the scoring step.

Listing 6: LSTM autoencoder (excerpt of `src/bearing_efw/models.py`).

```

1 class Net(nn.Module):
2     def __init__(self):
3         super().__init__()
4         self.enc = nn.LSTM(n_features, hidden, batch_first=True)
5         self.to_latent = nn.Linear(hidden, latent)
6         self.from_latent = nn.Linear(latent, hidden)
7         self.dec = nn.LSTM(hidden, hidden, batch_first=True)
8         self.out = nn.Linear(hidden, n_features)
9
10    def forward(self, x):
11        # x: (batch, window, features)
12        _, (h, _) = self.enc(x)
13        z = self.to_latent(h[-1])
14        # latent code
15        rep = self.from_latent(z).unsqueeze(1).repeat(1, x.shape[1], 1)
16        y, _ = self.dec(rep)
17        return self.out(y)
18
19    def fit(self, Xs):
20        # Windows are built per bearing so a sequence never spans two bearings.
21        W = torch.tensor(np.concatenate([self._windows(X)[self.window - 1:] for X in Xs]))
22        opt = torch.optim.Adam(self.net.parameters(), lr=self.lr)
23        for _ in range(self.epochs):

```

```

22     for idx in np.array_split(self.rng.permutation(len(W)), max(1, len(W) //
self.batch_size)):
23         xb = W[idx]
24         # denoising objective: reconstruct the clean window from a noisy copy
25         loss = ((self.net(xb + self.noise * torch.randn_like(xb)) - xb) ** 2).mean()
26         opt.zero_grad(); loss.backward(); opt.step()
27
28 def score(self, X):
29     with torch.no_grad():
30         W = torch.tensor(self._windows(X)) # window ending at each snapshot
31         return ((self.net(W) - W) ** 2).mean(dim=(1, 2)).numpy()

```

Line by line. Lines 4–8 create the building blocks: the encoder LSTM, two small linear layers that go into and out of the 8-number summary (the *latent code*), the decoder LSTM, and a final layer that outputs the rebuilt features. Lines 10–15 describe how data flows: the encoder reads the window (line 11), its final memory is squeezed into the summary z (line 12), the summary is copied once for each of the 12 time steps (line 13), and the decoder rebuilds the window from it (lines 14–15). In `fit`, line 19 cuts each bearing’s healthy data into 12-snapshot windows (never mixing two bearings). Line 21 repeats training 60 times (epochs); line 22 shuffles the windows and splits them into batches of about 64. Line 25 is the heart of the method: add noise to the batch, rebuild it, and measure the average squared mistake against the *clean* batch. Line 26 turns the knobs to reduce that mistake. In `score`, line 29 switches training off, line 30 builds the window that ends at each snapshot, and line 31 returns the average squared mistake for each window, which is exactly the formula e_t above.

5.5 From scores to alarms

In plain words

A score alone is just a number; an operator needs a clear “alarm” or “no alarm”. Three rules turn scores into alarms. (1) *Threshold*: from the calibration period we learn how high a healthy score can get, and add some headroom; above this line a score is “suspicious”. (2) *Debouncing*: one suspicious recording is not enough, because single blips happen. An alarm is raised only if at least 4 of the last 6 recordings (one hour) are suspicious, much as you would only call the plumber if the dripping sound comes back again and again. (3) *Persistence*: real damage does not heal by itself, so the warning we report is the final alarm that stays on until the end.

Threshold. For each bearing and detector, the threshold is derived from the calibration scores s only:

$$\tau = Q_{0.995}(s) + m (Q_{0.995}(s) - \text{median}(s)), \quad m = 0.5. \quad (8)$$

What the symbols mean. τ (the Greek letter “tau”) is the alarm threshold. s stands for all scores of the calibration period. $Q_{0.995}(s)$ is the 99.5% *quantile*: the value that only 0.5% of calibration scores (1 in 200) exceed. The median is the middle value (half of the scores are below it, half above). m is a safety margin factor, here 0.5.

What the formula does. Start from “almost the highest healthy score” and add extra headroom equal to half of the gap between that value and a typical healthy score. If healthy scores wander a lot, the gap is large and so is the headroom; if they are steady, the headroom is small.

The margin is expressed relative to the spread of the healthy scores, so the rule works for any score scale and sign (the RMS z-score can be negative, the other scores cannot).

Debouncing. A single noisy snapshot above τ should not page an operator. An alarm is active at snapshot t when at least $k = 4$ of the last $n = 6$ snapshots (one hour) exceed τ . This is called a k -of- n rule.

Persistent alarm. A real degradation does not heal. The warning reported for a failed bearing is the start of the *last* alarm episode (an uninterrupted stretch during which the alarm is on), provided it lasts until the end of the record; earlier episodes that cleared are counted separately.

What the code below does. It contains the three rules above: computing the threshold, applying the 4-of-6 rule, and finding the start of the final, lasting alarm.

Listing 7: Threshold, k-of-n debouncing and persistent alarm (src/bearing_efw/alarm.py).

```

1 def fit_threshold(calib_scores, quantile, margin):
2     q, med = np.quantile(calib_scores, quantile), np.median(calib_scores)
3     return float(q + margin * (q - med))
4
5 def k_of_n(exceed: np.ndarray, k: int, n: int) -> np.ndarray:
6     """Alarm at t when at least k of the last n snapshots (inclusive) exceed the threshold."""
7     c = np.convolve(exceed.astype(int), np.ones(n, dtype=int))[: len(exceed)]
8     return c >= k
9
10 def first_persistent_alarm(alarm, hours, start_h):
11     """Start of the final alarm episode, if it lasts until the end of the record."""
12     eps = [(s, e) for s, e in alarm_episodes(alarm) if hours[s] >= start_h]
13     if not eps:
14         return None
15     return float(hours[eps[-1][0]]) if eps[-1][1] == len(alarm) else None

```

Line by line. Lines 1–3 are the threshold formula, word for word. In `k_of_n`, `exceed` is a list of yes/no values (“was this recording above the threshold?”). Line 7 turns yes/no into 1/0 and, using `np.convolve` with a row of six ones, counts for every recording how many of the last six were “yes” (a sliding counter). Line 8 says “alarm” wherever this count is at least 4. In `first_persistent_alarm`, line 12 lists all alarm episodes that start after the calibration period; line 15 returns the start time of the last one, but only if it is still on at the very end of the record; otherwise there is no persistent alarm (`None`).

5.6 Diagnosis

In plain words

Once a lasting alarm is raised, the system asks: “which knocking rhythm has grown the most?” It looks at the outer-ring, inner-ring and roller rhythm features over the next 12 recordings (about two hours) and picks the one that stands out most. It only names a defect if the evidence is clear (more than 3 normal wiggles above its usual level); otherwise it honestly says “no clear defect frequency”, like a mechanic who will not guess the cause until the symptoms are clear.

When a persistent alarm is raised, the system looks at the standardized envelope features of the alarming bearing over the next 12 snapshots and reports the defect frequency with the largest average z-score, if it exceeds 3 (Listing 8). Otherwise it reports “no clear defect frequency” rather than guessing.

What the code below does. It implements this rule for one bearing.

Listing 8: Envelope-based diagnosis (excerpt of scripts/run_experiment.py).

```

1 DEFECTS = {"env_bpfo": "outer race", "env_bpfi": "inner race", "env_bsf": "rolling element"}

```

```

2
3 def diagnose(d, alarm_h, n=12):
4     if alarm_h is None:
5         return ""
6     cols = list(d.raw.columns)
7     after = np.flatnonzero(d.hours >= alarm_h)[:n]
8     z = {k: np.mean([d.X[after, cols.index(c)].mean() for c in cols if c.startswith(k + "_")])
9           for k in DEFECTS}
10    best = max(z, key=z.get)
11    return f"{DEFECTS[best]} (z={z[best]:.1f})" if z[best] > 3 else "no clear defect frequency"

```

Line by line. Line 1 links each rhythm feature to the part of the bearing it points to. Lines 4–5: if there is no alarm, there is nothing to diagnose. Line 7 picks the 12 recordings starting at the alarm. Lines 8–9 compute, for each of the three rhythm features, its average z-score over those recordings (and over both sensors in test 1). Line 10 picks the largest, and line 11 reports it (with its z-score) only if it is above 3.

5.7 Evaluation protocol

In plain words

This is how we grade the system. Since nobody knows exactly when the damage started, we use two simple marks. *Lead time*: how many hours before the end of the test the lasting alarm began; more is better, because it gives more time to plan a repair. *False alarms*: how often the system cried wolf during a stretch of time that we believe was still healthy. To compare tests of different lengths, false alarms are counted per 1 000 *bearing-hours* (one bearing watched for one hour is one bearing-hour; four bearings watched for 250 hours make 1 000). All the settings of the system were chosen by looking at test 2 only; tests 1 and 3 were kept aside like exam questions kept secret until the exam, so their results show how the system would do on a machine it has never seen.

The dataset only states that a bearing had failed when each test was stopped; the onset of degradation is not labelled. Two metrics follow from this:

- **Lead time** of a failed bearing: hours between its persistent alarm and the end of the test (also reported as a percentage of the test duration). Higher is better.
- **False alarms**: alarm episodes on *any* bearing inside a window that is believed to be healthy: after calibration (35 % of snapshots) and before 60 %, 50 % and 85 % of the snapshots in tests 1, 2 and 3 respectively. These limits come from inspecting the condition indicators (Section 6) and are documented in the configuration file. The rate is normalised per 1 000 bearing-hours of that window.

Hyper-parameters (settings chosen by a person rather than learned from data: split fractions, threshold quantile and margin, k and n , network size, epochs, noise) were chosen on test 2 only. Tests 1 and 3 were not used for any decision and act as held-out evaluations.

5.8 Design decisions

In plain words

These are the house rules that keep the study honest. The biggest danger in this kind of work is *data leakage*: information from the future (for example, from the period when the bearing is already breaking) sneaking into the learning step. That is like seeing the answers before an exam: the grade looks great but means nothing. The rules below prevent this, make sure we

watch the right bearings, treat loud and quiet bearings fairly, compare all detectors under the same rules, and make every run give exactly the same results.

- **Train on the beginning, test on the rest.** All splits are in time order. Shuffling windows from the whole run leaks the failure into training.
- **One snapshot is one point in time.** Features are computed per file, so results are in hours before failure.
- **Monitor the right bearing, and every bearing.** Each bearing is scored separately; the failed bearings are known from the dataset documentation and are only used for evaluation.
- **Split by snapshot count, not wall-clock time.** With test 1’s multi-day gaps, the first 25 % of the *time* would contain very few snapshots.
- **Per-bearing baselines.** Standardising each bearing against itself makes one model serve bearings with different natural vibration levels.
- **Same threshold rule and alarm logic for all detectors,** so differences in the results come from the detectors and not from tuning.
- **Determinism.** All random components are seeded (given a fixed starting point for their random numbers); two consecutive runs produce identical result tables.

6 Results

6.1 Headline results

In plain words: how to read the results table

Table 5 has one row per detector. The first four number columns are the four bearings that actually failed (“B3 inner” means bearing 3, which had an inner-ring defect). Each cell is the **lead time in hours**: how long before the end of the test the lasting alarm was raised. For example, the cell **74.3** in the row “RMS threshold” and the column “test2” means: *the RMS method raised an alarm that stayed on from then on, about 74 hours (roughly 3 days) before the bearing failed*. A bigger number is better. Bold marks the best value in each column.

The last column is the **false-alarm rate** per 1 000 bearing-hours. 1 000 bearing-hours is, for example, a machine with four bearings watched for 250 hours, i.e. about ten days. So **5.34** for the RMS method means about five unwarranted alarms in ten days of normal operation, roughly one every two days; **0.00** for Isolation Forest means no false alarm at all; and **12.67** for the LSTM autoencoder means a little more than one false alarm per day.

Why there is a trade-off. Think of a smoke detector. A very sensitive one notices a fire early, but it also goes off every time you burn the toast. A less sensitive one stays quiet during cooking, but it may only react when the fire is already large. Our detectors behave the same way: a setting or method that warns earlier usually also reacts to harmless changes, and one that is very quiet tends to warn later. A smaller table (Table 6) gives the raw count of false alarms for each test.

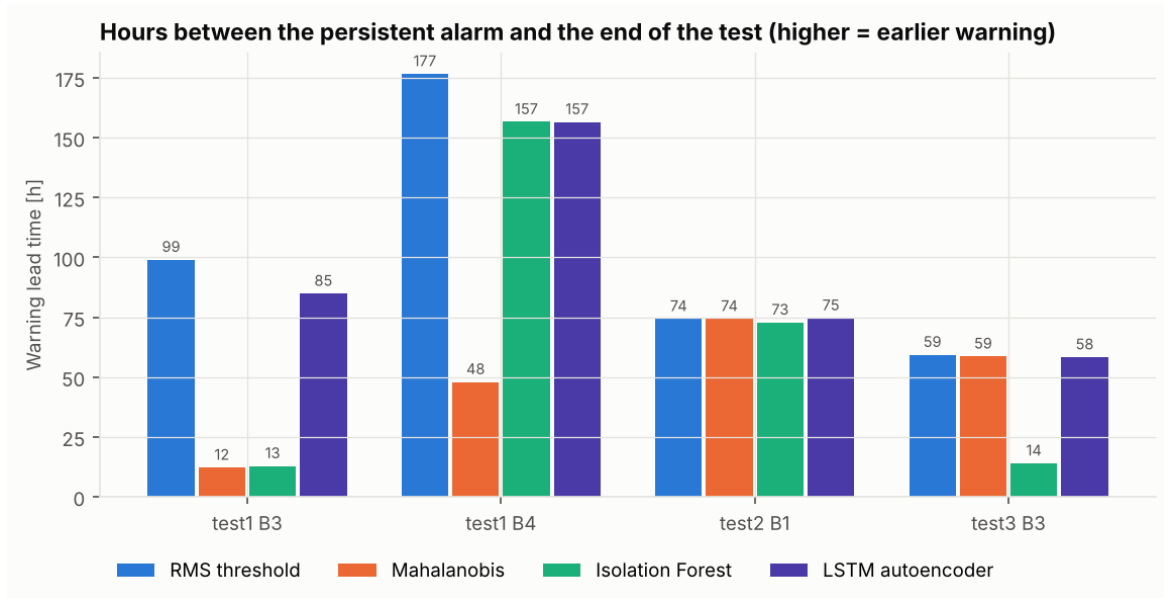
Table 5: Lead time of the persistent alarm on each failed bearing (hours before end of test) and false alarms. Bold: best value per column.

Detector	test1 (held out)		test2 (dev)	test3 (held out)	False alarms per 1000 bearing-h
	B3 inner	B4 roller	B1 outer	B3 outer	
RMS threshold	99.0	176.6	74.3	59.3	5.34
Mahalanobis	12.2	48.0	74.3	58.7	0.67
Isolation Forest	12.9	156.6	72.8	14.0	0.00
LSTM autoencoder	84.9	156.6	74.7	58.3	12.67

Table 6: False-alarm episodes per test and size of the evaluation window.

Detector	test1	test2	test3
RMS threshold	15	0	1
Mahalanobis	0	0	2
Isolation Forest	0	0	0
LSTM autoencoder	4	2	32
Healthy window (hours)	434–632	57–82	380–908
Healthy window (bearing-hours)	790	97	2 111

How to read Table 6. Each cell counts separate false-alarm episodes. For example, the RMS method raised 15 false alarms in test 1, and the LSTM autoencoder 32 in test 3. The bottom rows show the stretch of time (in hours since the start of each test) that was assumed to be healthy and used for this count.

**Figure 5:** Lead time of the persistent alarm for each failed bearing and detector.

How to read this figure

This is a grouped bar chart of the same numbers as Table 5. Along the horizontal axis are the four bearings that failed (for example, “test1 B3” is bearing 3 of test 1). Each group has four bars, one per detector: **blue** = RMS threshold, **orange** = Mahalanobis, **green** = Isolation Forest, **purple** = LSTM autoencoder. The vertical axis is the warning lead time in hours; the

number above each bar is the value, rounded. A taller bar means an earlier warning.

What do we see? On test 2 the four bars are practically equal (73–75 h). The held-out tests separate the detectors. On test 1 the RMS baseline and the LSTM autoencoder warn earliest (85–177 h), while Mahalanobis distance warns only 12–48 h before the end. On test 3 three detectors give about 59 h and Isolation Forest only 14 h.

What does it mean? When a defect develops cleanly, every reasonable detector sees it at the same moment, and the choice of detector does not matter. On messier data the choice matters a lot.

Why does it matter? Read together with Table 6, the detectors that warn earliest are also those that raise the most false alarms; there is no free lunch (no method is best at everything at once).

6.2 Test 2 (development set)

In plain words

Test 2 is the “practice” experiment used to choose the settings. For each test we show two figures. The first shows a few of the summary numbers (features) over time, so you can see with your own eyes when the bearing started to change. The second shows the score of each detector over time and when the alarm was raised.

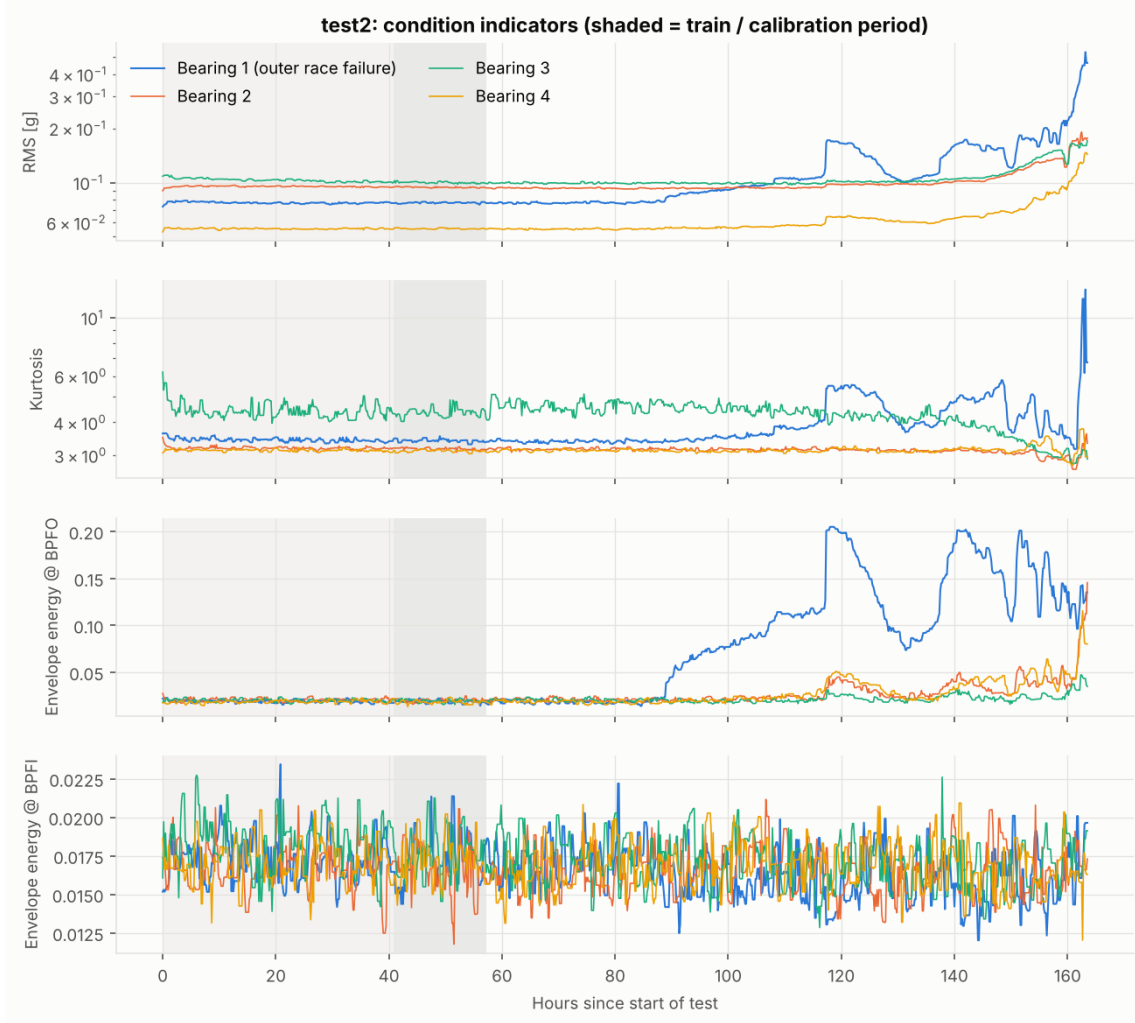


Figure 6: Condition indicators of test 2 (5-snapshot rolling median). Shaded: training and calibration periods.

How to read this figure

Four panels are stacked and share the same horizontal axis: **hours since the start of the test**. Each panel shows one indicator for all four bearings: **blue** = bearing 1 (the one that failed, with an outer-ring defect), **orange-red** = bearing 2, **green** = bearing 3, **amber** = bearing 4. From top to bottom the panels are: (1) **RMS** in g , the overall vibration strength; (2) **kurtosis**, the spikiness (about 3 = calm random vibration); (3) **envelope energy at BPFO**, the share of knocking rhythm at the outer-ring fingerprint; (4) **envelope energy at BPFI**, the same for the inner-ring fingerprint. The top two panels use a **log scale** (each labelled step is a multiplication, not an addition), the bottom two an ordinary scale. The **light grey shading** on the left is the training period and the **slightly darker grey** next to it is the calibration period. Each line is smoothed with a 5-snapshot rolling median (every point is the middle value of 5 consecutive recordings, i.e. of about 50 minutes), which removes isolated spikes.

What do we see? For the first 85 hours all four bearings are flat in every indicator. Around hour 88 the BPFO envelope energy of bearing 1 jumps from 0.02 to 0.05 and keeps rising to about 0.2 at hour 118, while its RMS increases only slowly. Bearing 1's kurtosis also rises (impacts) but more gradually. The BPFI envelope energy stays at noise level for all bearings. From about hour 115 the other bearings' RMS and BPFO energy also increase. In the last hours all indicators rise steeply. Bearing 1's BPFO energy shows a "dip" around hour 130.

What does it mean? The jump at hour 88 is the outer-race defect seen in Figure 4. The flat BPFI line is consistent with an outer-race (not inner-race) defect. The later rise on the other bearings happens because vibration from bearing 1 travels through the shaft and housing to its neighbours. The “dip” of bearing 1’s BPFO energy around hour 130 is typical of spall evolution: as the defect edges are worn smooth the impacts temporarily soften before the damage spreads.

Why does it matter? The rhythm feature reveals the problem long before the overall vibration level (RMS) becomes large, which is what makes an *early* warning possible.

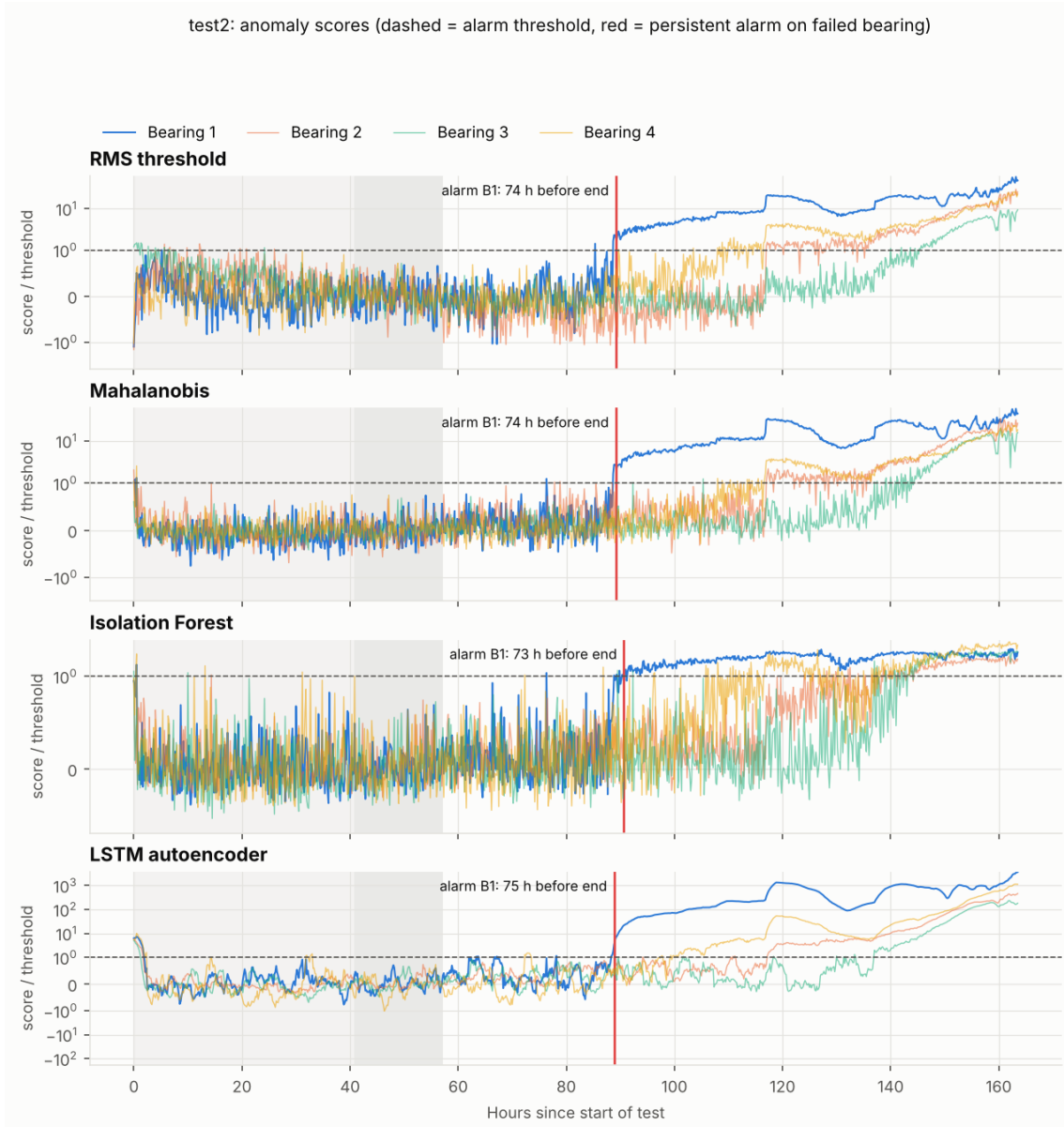


Figure 7: Test 2 anomaly scores divided by each bearing’s threshold (symmetric log scale). Dashed: alarm threshold; red line: persistent alarm on the failed bearing.

How to read this figure

One panel per detector (from top: RMS threshold, Mahalanobis, Isolation Forest, LSTM autoencoder). The horizontal axis is hours since the start of the test. The vertical axis is each bearing’s score *divided by its own alarm threshold*, so that all bearings can share one picture: a value of 1 means “exactly at the threshold”, above 1 means “above the threshold”. The **grey dashed horizontal line** marks this value 1. The vertical axis uses a **symmetric log scale**:

close to zero it is an ordinary scale, further away it becomes a log scale ($0, 10^0 = 1, 10^1 = 10, 10^2 = 100, \dots$, and the same mirrored for negative values, such as $-10^0 = -1$). This lets small wiggles near zero and enormous jumps fit in the same panel. Negative values occur only for the RMS method, whose score is a z-score and is below zero when the vibration is quieter than average. Colours as before (**blue** = bearing 1, the failed one). Grey shading: training and calibration periods. The **red vertical line** marks the start of the persistent alarm on the failed bearing, and the label next to it gives the lead time.

What do we see? All four detectors keep every bearing below the threshold during training, calibration and the following 30 hours. Then bearing 1 crosses the threshold at hour 89 and never returns: a persistent alarm 74h before the end. The score jump is largest for the LSTM autoencoder (two orders of magnitude, i.e. about a hundred times) and the Mahalanobis distance (about $20\times$), which combine all features, and smallest for Isolation Forest, whose score saturates by construction (it cannot grow beyond a certain level). Apart from two short LSTM episodes (which count as false alarms), the other bearings cross the threshold only after hour 109–145, always after bearing 1.

What does it mean? The *first bearing to alarm* correctly localises the source. All detectors diagnose an **outer race** defect at the alarm (z-scores 6.1–7.3), which is the documented failure mode. Bearings 2 and 4 receive an outer-race diagnosis too at later alarms, because they pick up bearing 1's vibration.

Why does it matter? This is why the dashboard marks the first bearing to alarm as the likely source: on its own, a later alarm on a neighbour could send the repair team to the wrong bearing.

6.3 Test 1 (held out)



Figure 8: Condition indicators of test 1. Line breaks are recording gaps.

How to read this figure

Same layout as Figure 6: four panels (RMS, kurtosis, envelope energy at BPFO and at BPFI) against hours since the start of the test. Here the bearings that failed are **bearing 3** (green, inner-ring defect) and **bearing 4** (amber, roller defect). The **empty stretches** where lines are interrupted are periods when nothing was recorded. The light grey shading (training) runs up to hour 290 and the darker grey (calibration) up to about hour 434.

What do we see? Test 1 is the most difficult record: it is fragmented by gaps of up to six days, and its training period (the first 539 snapshots, up to hour 290) contains several step changes between recording bursts. The degradation of the two failed bearings is clearly visible in kurtosis: bearing 4 becomes impulsive from about hour 680 (kurtosis up to 20) and bearing 3 from about hour 720 (up to 40), long before their RMS rises. The envelope energy at BPFI (bearing 3’s inner-race defect) does not rise at all, and BPFO energy of bearing 4 only fluctuates.

What does it mean? Because of the step changes, “healthy” is not a single stable state in test 1, which makes the system’s job harder. Spikiness (kurtosis) reveals both defects, but the rhythm features do not. The reason is physical: inner-race and rolling-element defects rotate in and out of the load zone (the part of the bearing that carries the weight), which spreads their envelope energy over sidebands (extra peaks just beside the main rhythm) at $\pm f_r$ and the cage frequency

rather than concentrating it at one line; a ± 3 Hz window around the harmonics misses most of it.

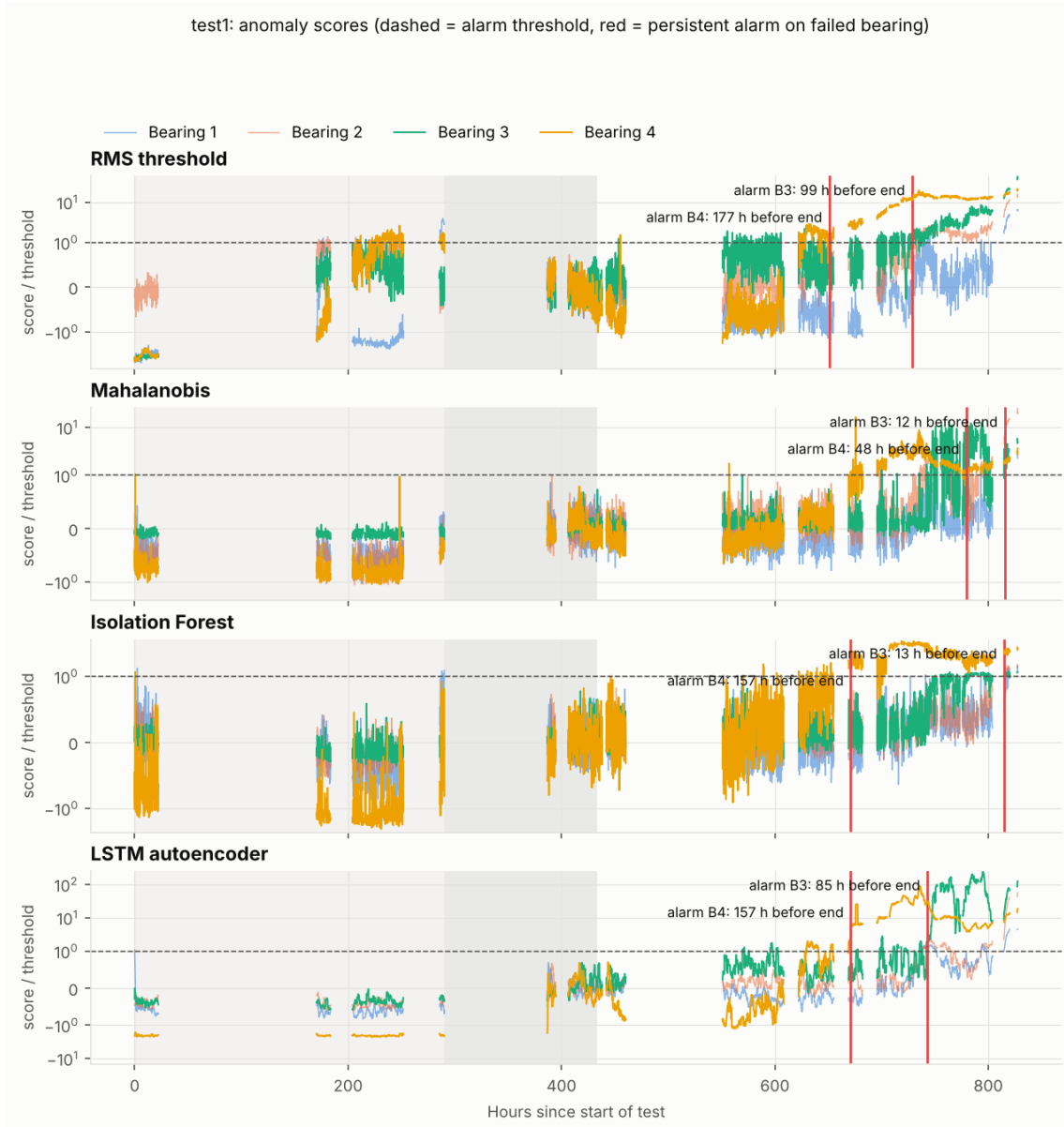


Figure 9: Test 1 anomaly scores divided by each bearing's threshold.

How to read this figure

Same layout as Figure 7: one panel per detector, score divided by threshold on a symmetric log scale, dashed line at 1 = threshold, grey shading = training and calibration. The two failed bearings (**green** = bearing 3, **amber** = bearing 4) are drawn in strong colours; the two healthy bearings (bearings 1 and 2) are drawn faintly. There are now **two red vertical lines** per panel, one for the persistent alarm on each failed bearing, each labelled with its lead time.

What do we see? Bearing 4 is detected by three detectors 157–177 hours before the end (19–21 % of the run), and both failed bearings are the first to alarm for every detector. The RMS baseline gives the earliest warnings but oscillates around its threshold inside the assumed-healthy window (hours 434–632), producing 15 alarm episodes on bearings 3 (from hour 554) and 4 (from hour 623). Mahalanobis distance and Isolation Forest raise none, but their persistent alarm on bearing 3 starts only 12–13 h before the end: they also crossed the threshold much earlier (bearing 3 first alarms at 747 and 756 h) but the score dipped back below it. No detector produces a confident

defect diagnosis on test 1.

What does it mean? Localisation is correct: the first bearings to alarm are the ones that failed. Because the 15 RMS episodes are exactly on the two bearings that later fail, some of these episodes may in fact be the first signs of degradation rather than false alarms; with no labelled onset, the evaluation conservatively counts them as false. For Mahalanobis and Isolation Forest, the conservative “persistent” definition discards their earlier warnings because the score dipped back. The LSTM autoencoder is the best compromise on this test (85 h and 157 h, 4 false-alarm episodes). The missing diagnosis has the reason given above (the narrow rhythm windows miss inner-race and roller defects).

Why does it matter? On a machine the system has never seen before, it still warned days in advance and pointed to the right bearings, but different detectors made very different trade-offs between early warning and false alarms.

6.4 Test 3 (held out)



Figure 10: Condition indicators of test 3.

How to read this figure

Same layout as Figure 6. The bearing that failed is **bearing 3** (green, outer-ring defect). The test lasted more than 1 000 hours; training (light grey) ends at about hour 268 and calibration (darker grey) at about hour 380. The small breaks in the lines are the machine stops (around hours 262, 354 and 1 002).

What do we see? Test 3 shows almost no sign of degradation for 1 000 hours. Two events matter. First, after the 7.5-hour stop around hour 354 the RMS of every bearing shifts slightly and bearing 4 shows a short spike at about hour 400. Second, after the 7.3-hour stop at hour 1 002, all indicators change, and from about hour 1 040 the BPFO envelope energy of bearing 3 (and, through transmission, of bearing 4) rises sharply, followed by RMS and kurtosis in the last 20 hours.

What does it mean? After the first stop the machine restarted in a slightly different condition. The outer-race defect on bearing 3 becomes visible only in the last 5 % of a 44-day run, so even a perfect detector could not have warned much earlier on this test.



Figure 11: Test 3 anomaly scores divided by each bearing's threshold.

How to read this figure

Same layout as Figure 7. The failed bearing (**green**, bearing 3) is drawn strongly, the other three faintly. The **red vertical line** marks the start of the persistent alarm on bearing 3, labelled with its lead time.

What do we see? The RMS, Mahalanobis and LSTM detectors raise a persistent alarm on bearing 3 about 59 h before the end, but this alarm starts about 5 hours after recording resumed following the stop (hour 1 009) and coincides with alarms on bearing 1. Isolation Forest fires only 14 h before the end, when the BPFO energy surges, and diagnoses an **outer race** defect with a z-score of 11.4. The LSTM autoencoder raises 32 false-alarm episodes, mostly on bearing 1, after hour 354.

What does it mean? Part of the 59-hour alarm is a reaction to the restart rather than to the defect, and these three detectors cannot tell which bearing is failing (bearings 1 and 3 alarm within the same snapshot). Isolation Forest is the only detector whose alarm is unambiguously caused by the defect. The regime shift (a lasting change in the machine’s normal operating condition) after hour 354 is the main source of false alarms: the LSTM autoencoder, which models the joint evolution of all features, interprets the new baseline as anomalous.

Why does it matter? This is the most important limitation of a model trained once on the beginning of the run: when the machine’s “normal” changes, for example after a stop or maintenance, the model keeps comparing against the old normal and may raise alarms that have nothing to do with damage.

6.5 Summary of findings

1. **Early warning works when the defect develops cleanly.** On test 2 all detectors warn 74 h (45 % of the run) before the end and name the correct defect.
2. **Lead time and false alarms trade off.** The RMS baseline and the LSTM autoencoder warn earliest; Isolation Forest never raised a false alarm but warns late on two of four failures; Mahalanobis distance sits in between with very few false alarms.
3. **A strong baseline matters.** The plain RMS threshold is competitive on lead time. The deep model does not dominate on a dataset with only three runs.
4. **Localisation by first alarm is reliable** on tests 1 and 2 but not after the restart in test 3.
5. **Envelope diagnosis is reliable for outer-race defects** (tests 2 and 3) but does not recognise test 1’s inner-race and roller defects with the current narrow-band feature.
6. **Operating-condition changes** (restarts, maintenance) are the main threat to validity: a model fitted once on the beginning of life treats a new normal as an anomaly.

In plain words

The system works well when a defect grows steadily: about three days of warning and the right diagnosis. No single method is best: the ones that warn earliest also cry wolf more often. The simplest method (watching the overall vibration level) holds its own against the neural network. The first bearing to raise an alarm is usually the culprit, and the “knocking rhythm” reliably recognises outer-ring damage but not yet the other kinds. The biggest weakness is that a restart of the machine can look like damage.

7 Interactive dashboard

In plain words

A dashboard is a screen that shows the state of a machine at a glance, like the warning lights in a car. Ours is a small web page (built with Streamlit, a tool that turns a Python program into a web page). It replays a recorded test as if the data were arriving live, so you can watch the alarms appear in the order an operator would have seen them.

To show how the system would look to an operator, `app/dashboard.py` replays any test as if it were streaming from the machine (Figure 12). The user selects a test and a detector and presses *Play*; the dashboard advances snapshot by snapshot and shows:

- a status tile per bearing (healthy / above threshold / alarm) with a colour, icon and label, the score-to-threshold ratio, the time since the current alarm started, a *first to alarm* marker for the likely source, and the diagnosed defect;
- the anomaly-score chart of all bearings up to the current time, with the training and calibration periods shaded;
- the envelope z-scores at BPFO, BPFI and BSF for a selected bearing;
- an alarm log and, on request, the ground truth (which bearing failed and how).

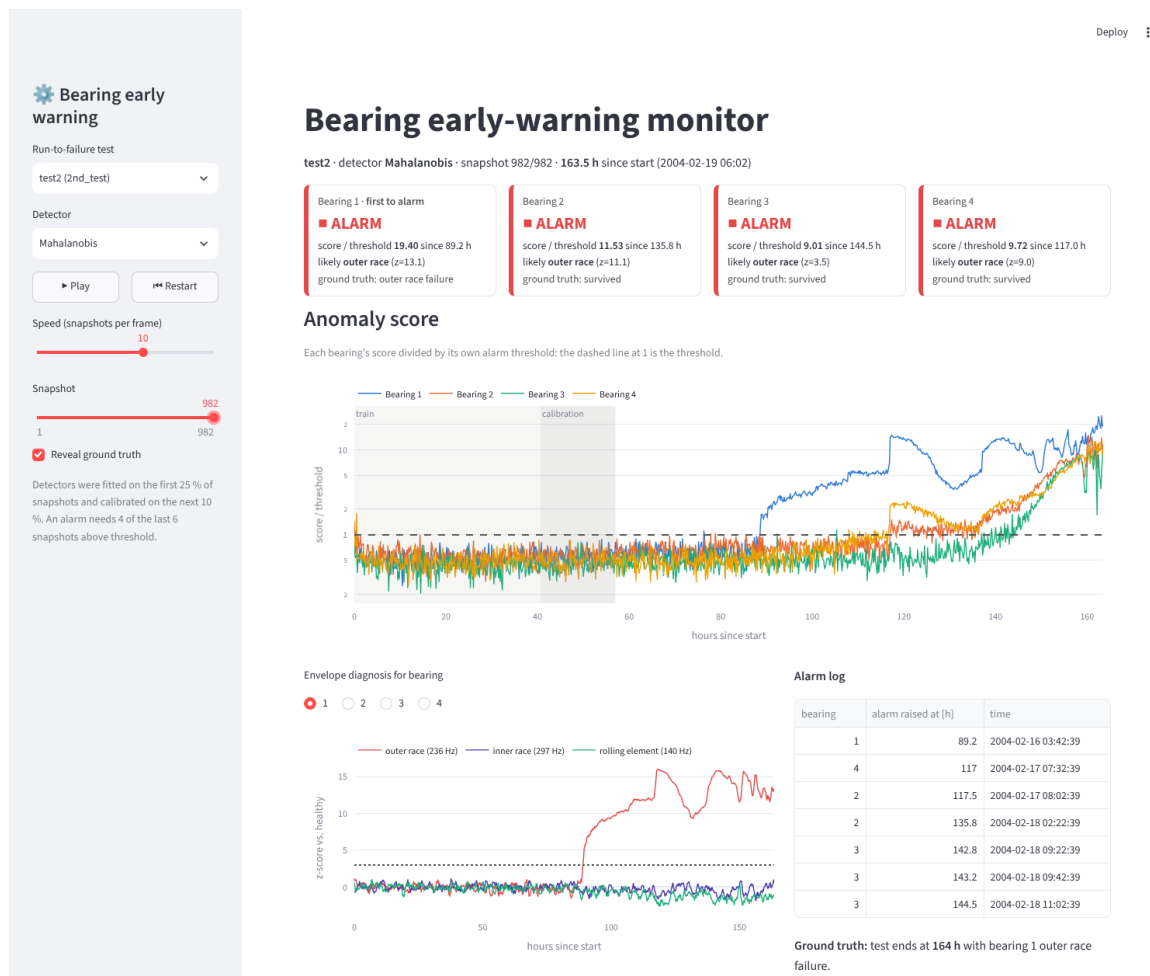


Figure 12: Dashboard at the end of test 2 with the Mahalanobis detector and ground truth revealed.

How to read this figure

This is a screenshot of the dashboard. **Left side panel:** the controls (choice of test and detector, Play and Restart buttons, playback speed, a slider to jump to any recording, and a checkbox to reveal the ground truth, i.e. what really happened). **Top row:** one tile per bearing, showing its status (here all four show a red ALARM), the score divided by the threshold, since which hour the alarm has been on, the likely defect with its z-score, and the ground truth. **Middle chart:** every bearing's score divided by its threshold over time (log scale); the dashed line at 1 is the threshold and the shaded areas are the training and

calibration periods. **Bottom left:** the envelope diagnosis for the selected bearing, as z-scores over time: **red** = outer race (236 Hz), **purple** = inner race (297 Hz), **green** = rolling element (140 Hz); the dotted horizontal line at 3 is the level a rhythm must exceed before a defect is named. **Bottom right:** the alarm log, a list of when each alarm was raised.

What do we see? The tiles summarise the story of test 2 in one line: bearing 1 is the first to alarm (since hour 89.2) with a clear outer-race signature ($z = 13.1$), the others follow 28–55 hours later. The envelope chart shows the BPFO z-score of bearing 1 jumping from 0 to above 10 around hour 90 while BPF1 and BSF stay flat.

What does it mean? The evidence behind the diagnosis is visible to the operator: they do not have to trust a black box, they can see for themselves that the outer-ring rhythm has jumped.

What the code below does. It decides which bearing is the likely source, computes the status of each tile, and makes the “Play” button advance the replay.

Listing 9: Likely source and status of each bearing (excerpt of `app/dashboard.py`).

```

1  # The first bearing to raise an alarm is the likely source: once it degrades, its
2  # vibration travels through the shaft and housing and lifts its neighbours too.
3  onsets = {b: g.loc[g["alarm"], "hours"].min() for b, g in seen.groupby("bearing") if
4             g["alarm"].any()}
5  source = min(onsets, key=onsets.get) if onsets else None
6
7  for col, (b, g) in zip(cols, seen.groupby("bearing")):
8      last = g.iloc[-1]
9      state = "alarm" if last.alarm else ("watch" if last.score > last.threshold else "ok")
10     defect, zval = diagnose(zs[b].loc[:now])      # mean z of the last 6 snapshots
11     ...
12
13 # Playback: advance the slider and rerun the script
14 if st.session_state.playing and idx < n:
15     st.session_state["advance_to"] = min(n, idx + speed)
16     time.sleep(0.15)
17     st.rerun()

```

Line by line. Line 3 finds, for every bearing that has alarmed so far, the hour of its first alarm; line 4 picks the earliest one as the likely source. Lines 6–9 fill one tile per bearing: line 7 takes its latest recording, line 8 sets the status to “alarm”, “watch” (above the threshold but not yet 4 out of 6) or “ok”, and line 9 computes the likely defect from the rhythm features. Lines 13–16 make the replay move: if Play is on, jump ahead by the chosen speed, wait 0.15 seconds, and redraw the page.

The dashboard reads only the committed result files, so it runs without the raw dataset: `make dashboard` opens it at `http://localhost:8501`.

8 Software engineering and reproducibility

8.1 Repository layout

In plain words

The project’s files are organised like a well-labelled filing cabinet: settings in one drawer, the core program in another, scripts that run the experiment in a third, the dashboard, the data summaries, the results, the tests and this report each in their own place. The list below shows each folder or file on the left and what it contains on the right.

```

1  configs/default.yaml      splits, thresholds, alarm rule, model hyper-parameters

```

```

2 src/bearing_efw/
3   dataset.py          test metadata, bearing geometry, defect frequencies, loader
4   features.py         time, spectral and envelope features per snapshot
5   preprocess.py       per-bearing matrices, log + standardisation, splits
6   models.py           RMS, Mahalanobis, Isolation Forest, LSTM autoencoder
7   alarm.py            thresholding, k-of-n debouncing, alarm episodes
8   plots.py            report figures
9 scripts/              download_data, build_features, run_experiment, make_report_figures
10 app/dashboard.py      Streamlit replay dashboard
11 data/features/        extracted features (committed, 3 MB)
12 reports/              results.md, summary.csv, scores.csv.gz, figures/
13 notebooks/           01_walkthrough.ipynb
14 tests/                unit tests and dashboard smoke tests
15 report/               this report (LaTeX sources and PDFs, English and Turkish)

```

8.2 Configuration

In plain words

All the adjustable settings (how long the learning period is, how high the alarm line is, the 4-of-6 rule, the size of the neural network, and so on) are written in one small text file, like the settings menu of a phone. To try a different setting you edit this file; you do not have to touch the program.

Every experimental choice lives in one YAML file (a simple, human-readable settings format), so the experiment can be changed without touching code:

```

1 split:      {train_end: 0.25, calib_end: 0.35}
2 healthy_end: {test1: 0.60, test2: 0.50, test3: 0.85}
3 threshold:  {quantile: 0.995, margin: 0.5}
4 alarm:      {k: 4, n: 6}
5 models:
6   isolation_forest: {n_estimators: 300, seed: 0}
7   lstm_ae: {window: 12, hidden: 32, latent: 8, epochs: 60, lr: 0.002,
8             batch_size: 64, noise: 0.1, seed: 0}

```

Line by line. `split`: training ends at 25 % and calibration at 35 % of each bearing's recordings. `healthy_end`: the end of the stretch assumed healthy for counting false alarms (60 %, 50 % and 85 % of tests 1, 2 and 3). `threshold`: the 99.5 % quantile and the 0.5 margin from the threshold formula. `alarm`: the 4-of-6 rule. `models`: 300 trees for Isolation Forest, and the neural network's settings (window of 12 recordings, 32 memory units, an 8-number summary, 60 epochs, learning rate 0.002, batches of 64, noise level 0.1). `seed`: 0 fixes the random numbers so every run gives the same result.

8.3 Running

In plain words

These are the commands a user types to install and run everything. `make` is a shortcut tool: one short word such as `make experiment` runs a whole sequence of steps.

```

1 pip install -r requirements.txt
2 make experiment    # ~2 min, uses the committed feature tables
3 make dashboard    # interactive replay at http://localhost:8501
4 make test
5 make data          # optional: download + extract raw data (~1 GB / 6 GB)
6 make features      # optional: rebuild the feature tables from raw data

```

Line by line. The first line installs the required Python libraries. `make experiment` re-runs the whole study (about two minutes) from the saved summary tables. `make dashboard` opens the dashboard in a web browser. `make test` runs the automatic checks. The last two commands are optional: they download the 1 GB raw data and recompute the summary tables from it.

8.4 Testing

In plain words

Automatic tests are small programs that check that the main program still does the right thing, every time something is changed. The most important one is like a fire drill: we build an artificial vibration signal that contains fake knocks at exactly the outer-ring rhythm, hidden in random noise, and check that the system notices them and blames the outer ring. If this test ever fails, we know the rhythm feature is broken before it can produce wrong results.

Twelve automated tests check the parts most likely to break silently. The most important one builds a synthetic (artificially generated) outer-race defect (resonance bursts repeating exactly at BPFO, buried in noise) and checks that the envelope feature detects it and attributes it to the outer race (Listing 10). Other tests check the defect frequencies against the published values, the statistics of Gaussian noise (RMS = 1, kurtosis = 3), the debouncing and persistence logic, the threshold’s invariance to a score shift (the threshold moves along correctly if all scores are shifted by the same amount), and that the dashboard renders without errors for every test with two detectors.

What the code below does. It builds the fake faulty signal and a fake healthy signal, computes their features, and checks three expectations.

Listing 10: Synthetic fault test (excerpt of `tests/test_core.py`).

```

1 def test_envelope_picks_up_outer_race_impacts():
2     """Synthetic outer-race defect: resonance bursts repeating at BPFO."""
3     rng = np.random.default_rng(1)
4     t = np.arange(FS) / FS
5     bpfo = fault_frequencies()["bpfo"]
6     impacts = np.zeros(FS)
7     impacts[(np.arange(0, 1, 1 / bpfo) * FS).astype(int)] = 1.0
8     ring = np.exp(-t[:200] * 2000) * np.sin(2 * np.pi * 4000 * t[:200])
9     faulty = np.convolve(impacts, ring)[:FS] * 3 + 0.3 * rng.standard_normal(FS)
10    healthy = 0.3 * rng.standard_normal(FS)
11    f_faulty, f_healthy = snapshot_features(faulty), snapshot_features(healthy)
12    assert f_faulty["env_bpfo"] > 5 * f_healthy["env_bpfo"]
13    assert f_faulty["env_bpfo"] > f_faulty["env_bpfi"]
14    assert f_faulty["kurtosis"] > f_healthy["kurtosis"]

```

Line by line. Line 3 creates a random-number generator with a fixed seed. Line 4 creates a one-second time axis. Line 5 gets the outer-ring rhythm (236.4 Hz). Lines 6–7 place a “knock” at every 1/236.4 of a second. Line 8 designs a short ringing tone at 4000 Hz that dies away quickly, like a bell struck once. Line 9 makes each knock trigger that ringing and adds random noise: this is the fake faulty bearing. Line 10 is a fake healthy bearing (noise only). Line 11 computes the features of both. Lines 12–14 are the checks: the outer-ring rhythm feature must be more than five times larger for the faulty signal, it must beat the inner-ring rhythm feature, and the faulty signal must be spikier (higher kurtosis).

8.5 Version control

In plain words

Git keeps a complete history of every change to the project, like “track changes” in a word processor, and GitHub stores that history online. New features were developed on separate *branches* (parallel copies of the project) and then merged in through *pull requests* (proposed changes that can be reviewed before they are accepted).

The project is hosted on GitHub (https://github.com/ecemaydemir/bearing_early_fault_warning). The initial pipeline was committed to *main*; later additions (the dashboard, this report) were developed on separate branches and merged through pull requests.

9 Discussion and limitations

In plain words

No study is perfect, and this section lists honestly what could make the results less reliable. We do not know exactly when each defect started, so the grades are approximate. Three experiments are a small sample, so the ranking of the methods could change with more data. Restarting the machine changes its “normal”, which the system currently mistakes for damage. The rhythm feature recognises outer-ring damage but is too narrow for the other kinds. And our rule of only counting alarms that stay on until the end is strict: it throws away some warnings that were in fact early and correct.

- **Ground truth.** Only “failed at the end of the test” is known. Lead time therefore measures how early an alarm persists, not detection of a known onset, and the healthy windows used for false alarms are a documented judgement.
- **Small sample.** Three runs and four failed bearings are not enough for statistically robust rankings; numbers will move with other settings. Settings were chosen on test 2 only to keep the held-out results honest.
- **Non-stationary operation.** Restarts in test 3 move every bearing’s baseline (its “normal” level). In a real plant, the baseline would be re-estimated after maintenance or when an operator confirms that a change is benign.
- **Narrow-band diagnosis.** Inner-race and rolling-element defects are modulated by shaft and cage rotation (their knocks get louder and softer as the damaged spot moves in and out of the loaded zone); sideband-aware or spectral-kurtosis-based band selection (methods that also look at the side peaks, or that automatically find the pitch range where knocks are clearest) would make the diagnosis more general.
- **Persistence is conservative.** Requiring the alarm to persist until the end removes intermittent but real early warnings (test 1, bearing 3); in practice, an intermittent alarm is itself useful information for the operator.

10 Conclusion and future work

In plain words: the whole project in six sentences

Bearings are small parts that let machine shafts turn, and when they break the whole machine stops. We built a system that listens to a bearing’s vibration, learns what “healthy” sounds like from the first part of its life, and raises an alarm when the sound changes for good. On the cleanest experiment it warned about three days before the failure and correctly said that the outer ring was damaged. On two experiments it had never seen, it also warned in advance, but the methods that warn earliest also give more false alarms, just like an over-sensitive smoke detector. Its main weakness is that a restart of the machine can look like damage, so a real system would need to re-learn “normal” after maintenance. Everything is free, open and can be re-run with one command on an ordinary laptop.

The project delivers a working, reproducible early-warning pipeline for bearings that uses no failure labels. On the cleanest run it warns three days (45 % of the run) ahead and names the right defect; on the harder runs it makes the trade-off between lead time and false alarms explicit and documents where and why it fails. The most valuable next steps are:

1. adaptive re-baselining after restarts and maintenance (test 3), i.e. letting the system re-learn “normal” when the machine’s condition legitimately changes;
2. remaining-useful-life estimation after the alarm (predicting how many hours are left), e.g. by fitting a degradation trend to the health indicator;
3. sideband-aware envelope features or spectral kurtosis for inner-race and rolling-element diagnosis;
4. an ensemble alarm (e.g. two of four detectors must agree) to combine early warning with few false alarms;
5. validation on other run-to-failure datasets (e.g. FEMTO/PRONOSTIA, XJTU-SY).

A Glossary of Key Terms

Terms are grouped by topic. Each explanation is deliberately simple; the main text gives the precise technical meaning where needed.

The machine and its parts

Bearing (rolling-element bearing)

A part that lets a shaft turn smoothly with little friction. It has two rings with balls or rollers between them, like the wheel bearing of a car or a bicycle.

Inner race

The inner ring of a bearing. It sits on the shaft and turns with it.

Outer race

The outer ring of a bearing. It sits still in the machine’s housing.

Rolling element

One of the balls or rollers that roll between the two rings.

Cage

A light frame that keeps the rolling elements evenly spaced, like the spacer in a box of eggs.

Spall

A small flake of metal that breaks out of a bearing surface, leaving a pit. Each roller knocks as it passes over it, like a tyre over a pothole.

Run-to-failure

An experiment in which a machine is kept running, without repair, until a part breaks. It shows the whole life of a part, from new to broken.

rpm

Revolutions per minute: how many full turns the shaft makes in one minute. 2000 rpm is about 33 turns per second.

Measuring vibration**Vibration**

The fast back-and-forth shaking of a machine. A worn bearing changes the way a machine shakes, just as a sick person's heartbeat may sound different.

Accelerometer

A small sensor glued or screwed to the machine that measures how strongly it shakes, like the motion sensor in a phone that knows when you tilt it.

g (unit)

The unit of acceleration used here: 1 *g* is the acceleration of gravity, the pull you feel when you stand still. A vibration of 0.1 *g* is a tenth of that.

Hz (hertz)

"Times per second". A rhythm of 236 Hz repeats 236 times per second; 1 kHz is 1 000 times per second.

Sampling frequency

How many measurements the sensor takes per second. Here 20 480 per second, like a camera taking 20 480 photos per second.

Snapshot

One one-second recording of vibration (20 480 numbers per sensor). The test machine recorded one snapshot every 10 minutes.

Channel

One sensor's stream of measurements. Test 1 had two sensors per bearing, so eight channels in total.

Analysing signals**Frequency**

How often something repeats per second, measured in Hz. In sound it is the pitch: a high frequency is a high-pitched tone.

Spectrum

A breakdown of a signal into its frequencies, showing how much of each is present, like the bars of a music equaliser.

FFT (fast Fourier transform)

The standard, fast computer method for calculating a spectrum. It is like a prism that splits white light into its colours.

Resonance

The natural “ringing tone” of an object. When struck, a bell rings at its own note; a bearing housing does the same when a roller hits a defect.

Band-pass filter

A tool that keeps only a chosen range of frequencies and removes the rest, like tuning a radio to one station.

Envelope

The smooth outline that follows the peaks of a fast-shaking signal, i.e. how its loudness rises and falls over time.

Hilbert transform

The mathematical tool used to draw the envelope of a signal.

Envelope spectrum

The spectrum of the envelope. It shows the *rhythms* in the loudness, for example how many times per second a defect is struck.

BPFO, BPFI, BSF, FTF

The four “fingerprint” rhythms of a bearing, calculated from its size and speed: knocks per second from a defect on the outer ring (BPFO), on the inner ring (BPFI), on a roller (BSF), and the turning rate of the cage (FTF). If the envelope spectrum has a peak at one of them, that part is probably damaged.

Harmonic

A whole-number multiple of a frequency. The harmonics of 236 Hz are 473 Hz ($2\times$), 709 Hz ($3\times$), and so on. A regular knock produces peaks at all of them.

Sideband

A smaller peak just to the left and right of a main peak, caused by the main rhythm getting louder and softer at a slower pace.

Summary numbers (features)**Feature (condition indicator)**

One summary number computed from a recording, such as its average strength. Each recording is described by 18 features, like a blood test described by a few values.

RMS (root mean square)

The average strength of the vibration. It grows as the machine shakes more.

Peak

The largest single swing in a recording.

Kurtosis

A measure of spikiness. A smooth, random hum gives about 3; occasional sharp knocks give much higher values.

Skewness

Whether a signal swings further to one side of zero than to the other.

Crest factor

The largest swing divided by the average strength. A tall spike in an otherwise calm signal gives a high crest factor.

Spectral centroid

The “average pitch” of a recording, weighted by how strong each pitch is.

Logarithm

A way of rescaling numbers so that multiplying by 10 becomes adding one step (log of 10, 100, 1 000 grows evenly). It keeps very large values from dominating.

Log scale

A chart axis on which equal steps mean “ten times more” rather than “ten more”.

Standardization / z-score

Re-expressing a value as “how many normal wiggles (standard deviations) it is above or below the normal average”. A z-score of 3 means clearly higher than usual; 0 means perfectly average.

Standard deviation

The typical size of the ups and downs around an average.

Median

The middle value: half of the values are below it and half above.

Quantile

A cut-off value below which a given share of the data lies. The 99.5 % quantile is the value that only 1 in 200 observations exceeds.

Learning and detecting**Training period**

The first part (25 %) of each bearing’s life, assumed healthy, from which the system learns what “normal” looks like.

Calibration period

The next 10 %, used only to decide where to put the alarm threshold.

Test period

Everything after calibration: the system is “on duty” and its alarms are graded.

Unsupervised learning

Learning without being told the right answers. Here the computer is never told which recordings are faulty; it only learns what normal looks like.

Label

A note saying what state the data is in (for example “faulty”). This project uses no labels for learning.

Anomaly score

A number given to each recording: small if it looks like the healthy data, large if it looks unusual.

Threshold

The score above which a recording counts as suspicious, like the line on a thermometer above which you have a fever.

Detector

A method that learns “normal” and produces an anomaly score. Four are compared here.

Mahalanobis distance

A “how far from normal” measure that takes into account how much each feature usually varies and which features usually move together.

Covariance

A table that says how much each feature varies and how strongly pairs of features rise and fall together, for example height and weight in people.

Ledoit–Wolf

A method for estimating the covariance more reliably when there are few examples, by blending the measured table with a simpler, safer guess.

Isolation Forest

A detector that tries to single out each recording with random yes/no questions. Unusual recordings are singled out with fewer questions.

Neural network

A computer model made of many simple connected units with adjustable “knobs” (parameters), loosely inspired by the brain. It learns by turning these knobs to reduce its mistakes.

Parameter

One adjustable number inside a model. Training means finding good values for all of them.

LSTM (long short-term memory)

A kind of neural network that reads data in order and remembers what came before, so it can notice changes over time.

Autoencoder

A neural network that squeezes its input into a short summary and then rebuilds it. If it has only seen healthy data, it rebuilds unfamiliar data badly, and the size of the mistake is the anomaly score.

Denoising autoencoder

An autoencoder trained to rebuild a clean input from a slightly noisy copy, which forces it to learn general patterns instead of memorising.

Epoch

One full pass of the training process over all training examples.

Hyper-parameter

A setting that a person chooses before learning, such as the window length or the threshold margin, as opposed to the parameters that the model learns itself.

Seed

A fixed starting point for a computer’s random numbers, so that “random” steps give the same result every time.

Alarms and evaluation**k-of-n rule (debouncing)**

Raise an alarm only if at least k of the last n recordings are above the threshold (here 4 of 6). It stops single blips from triggering alarms, like waiting for a dripping sound to repeat before calling the plumber.

Alarm episode

One uninterrupted stretch of time during which the alarm is on.

Persistent alarm

An alarm episode that stays on until the end of the record. Real damage does not heal, so this is the warning that counts.

Lead time

How many hours before the end of the test the persistent alarm began: the time the operator would have had to plan a repair. More is better.

False alarm

An alarm raised while the machine was believed to be healthy: the system “crying wolf”.

Bearing-hour

One bearing watched for one hour. Four bearings watched for 250 hours give 1 000 bearing-hours; false alarms are counted per 1 000 bearing-hours to compare tests fairly.

Ground truth

What really happened (here: which bearing was found broken at the end, and how), used only to grade the system.

Development set / held-out set

The development set (test 2) is used to choose settings, like practice questions. The held-out sets (tests 1 and 3) are kept aside and used only once, like the real exam.

Data leakage

When information that would not be available in real life (for example, from the future or from the test data) sneaks into the learning step. It makes results look better than they really are, like seeing the answers before an exam.

Regime shift

A lasting change in a machine’s normal condition, for example after a stop and restart. The system may mistake the new normal for damage.

Localisation

Working out *which* bearing is the source of a problem. Here: the first bearing to raise an alarm.

Diagnosis

Working out *what kind* of damage it is (outer ring, inner ring or roller).

Software**Python**

The programming language the project is written in.

Library

A ready-made collection of program code that others have written and shared, such as NumPy for maths or PyTorch for neural networks.

Pipeline

The chain of processing steps from raw recordings to alarms, like an assembly line.

Dashboard

A screen that shows the state of a machine at a glance, like the warning lights in a car.

Streamlit

A free tool that turns a Python program into an interactive web page; it is used to build the dashboard.

Git / GitHub

Git records the full history of changes to the project’s files; GitHub stores that history online so others can see and use it.

Unit test

A small automatic check that one piece of the program still gives the expected answer.

References

- [1] J. Lee, H. Qiu, G. Yu, J. Lin, and Rexnord Technical Services. *Bearing Data Set*. IMS, University of Cincinnati. NASA Prognostics Data Repository, NASA Ames Research Center, 2007.
- [2] H. Qiu, J. Lee, J. Lin, and G. Yu. Wavelet filter-based weak signature detection method and its application on rolling element bearing prognostics. *Journal of Sound and Vibration*, 289(4–5):1066–1090, 2006.
- [3] R. B. Randall and J. Antoni. Rolling element bearing diagnostics—A tutorial. *Mechanical Systems and Signal Processing*, 25(2):485–520, 2011.
- [4] F. T. Liu, K. M. Ting, and Z.-H. Zhou. Isolation forest. In *Proc. IEEE International Conference on Data Mining*, pages 413–422, 2008.
- [5] O. Ledoit and M. Wolf. A well-conditioned estimator for large-dimensional covariance matrices. *Journal of Multivariate Analysis*, 88(2):365–411, 2004.
- [6] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [7] P. Malhotra et al. LSTM-based encoder-decoder for multi-sensor anomaly detection. *ICML Anomaly Detection Workshop*, 2016.